

第1章：企业级大模型的部署

讲师：尚硅谷-宋红康

欢迎访问尚硅谷官网 (<http://www.atguigu.com>) 获取更多学习资料

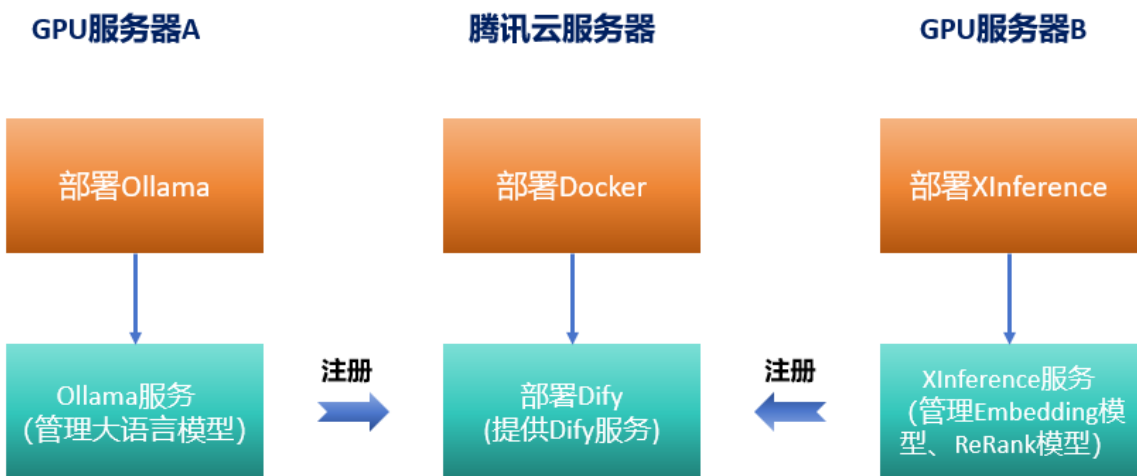
1、为什么需要企业级大模型的部署

1.1 整体部署方案

企业在使用大模型的过程中，首要保证的就是数据的安全。那要想数据安全，就需要实现私有化的部署。这里包括大语言模型、嵌入模型、重排序模型以及多模态模型等。

这里是咱们这套课程的整体部署方案。主服务器中安装Docker，部署Dify，对外统一提供大模型服务。比如：RAG、智能体Agent。同时，咱们也会结合不同的模型服务平台来满足多样化的需求，这里我们在GPU服务器A和B中，分别部署Ollama和Xinference，以便展示Dify管理多个外部模型服务的能力。进而再独立的部署大语言模型以及嵌入模型、重排序模型。

最后在Dify中调用前面配置好的大模型即可。



这里是我们配置或部署的三条主线。整体课程也是顺着这条主线展开：



主线1:	AutoDL	Ollama	deepseek / Qwen
主线2:	AutoDL	Xinference	Embedding / Rerank

1.2 问题答疑

针对上图，这里有几个问题。

问题1：这个项目中只把Dify安装在了Docker中。Ollama、Xinference直接部署在GPU服务器，没有在Docker中。为什么？

首先说，Ollama、Xinference是可以部署到Docker中的。但是Ollama、Xinference中模型使用，需要消耗大量GPU，而Docker中默认是使用CPU，不能使用GPU。所以：

方案1：Ollama、Xinference不安装在Docker中

方案2：Ollama、Xinference安装Docker中，但是需要额外安装其他的软件，支持GPU的调用。

这里使用方案1，只将Dify安装到docker中，Ollama和Xinference不安装到docker中。

问题2：为什么不在Ollama平台安装嵌入模型（Embedding Models）和重排序模型（ReRank Models）？

Ollama专注于简化大语言模型（LLM）的本地部署和管理。对嵌入模型（Embedding Models）和重排序模型（ReRank Models）的直接支持相对有限，可用的模型也较少（比如嵌入模型）有的甚至直接没有（比如重排序模型）。所以会选择XInference。

问题3：Xinference不了解，简单介绍一下？

Xinference是一个功能更全面的模型服务框架，它不仅支持部署和管理大语言模型，还对嵌入模型和重排序模型提供了广泛且灵活的支持。

问题4：为何考虑多服务器部署？

虽然将Ollama、Xinference与Dify部署在同一台服务器上可行的，但采用分离部署（即将Ollama和Xinference分别部署在独立的服务器上）通常能带来以下好处：

- **好处1：分散计算压力，优化资源利用**：大语言模型及相关AI模型的运算（尤其是推理过程）对GPU等计算资源消耗较大。所以把不同的模型服务部署在不同的服务器上，可以将计算负载分摊开，当特定模型被调用时，其计算压力将主要集中在承载该模型的服务器上，避免单一服务器遇到资源瓶颈。确保整体的稳定高效运行。
- **好处2：提高可扩展性和灵活性**：随着Dify平台上接管的应用增多或特定模型调用频率增加，可能需要独立扩展某个模型服务（如增加更多GPU资源给Xinference服务器）。多服务器架构为此类精细化的资源扩展提供了便利。
- **好处3：演示Dify的分布式模型管理能力**：此架构清晰地演示了Dify平台如何有效地管理部署在不同服务器上的Ollama和Xinference所提供的模型服务，有助于构建复杂的、分布式的AI应用。

2、企业级大模型的应用场景

场景1：基于RAG架构的开发

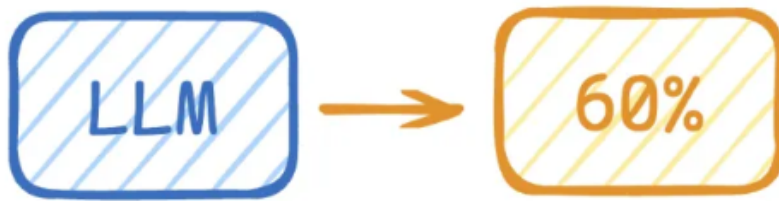
背景：

- 大模型的知识冻结
- 大模型幻觉

随着 LLM 规模扩大，训练成本与周期相应增加。因此，包含最新信息的数据难以融入模型训练过程，无法及时反映最新的信息或动态变化。导致 LLM 在应对诸如“请推荐当前热门影片”等时间敏感性问题。

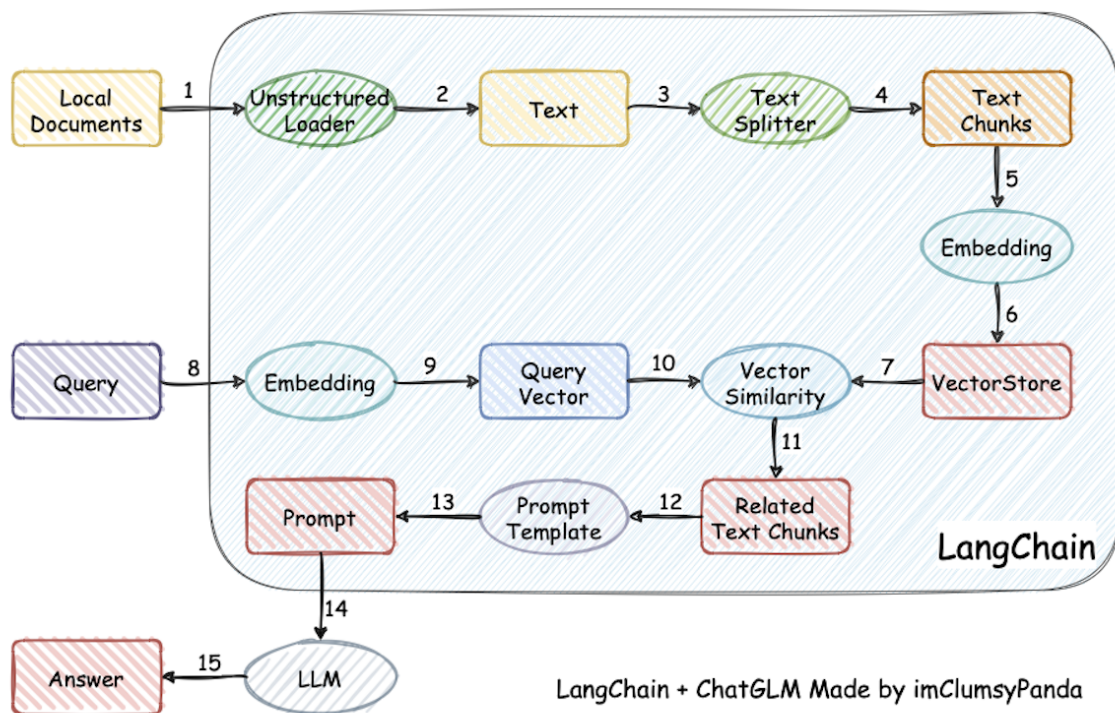
举例：

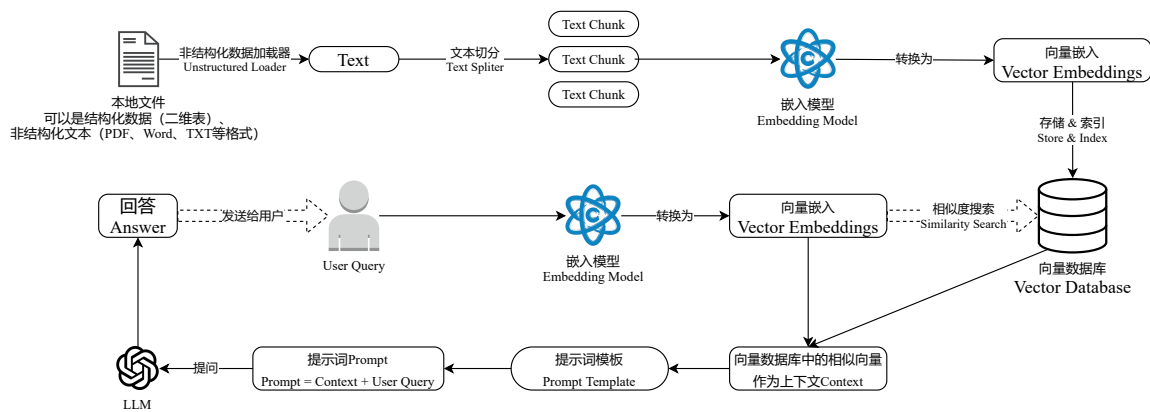
LLM在考试的时候面对陌生的领域，答复能力有限，然后就准备放飞自我了，而此时RAG给了一些提示和思路，让LLM懂了开始往这个提示的方向做，最终考试的正确率从60%到了90%！



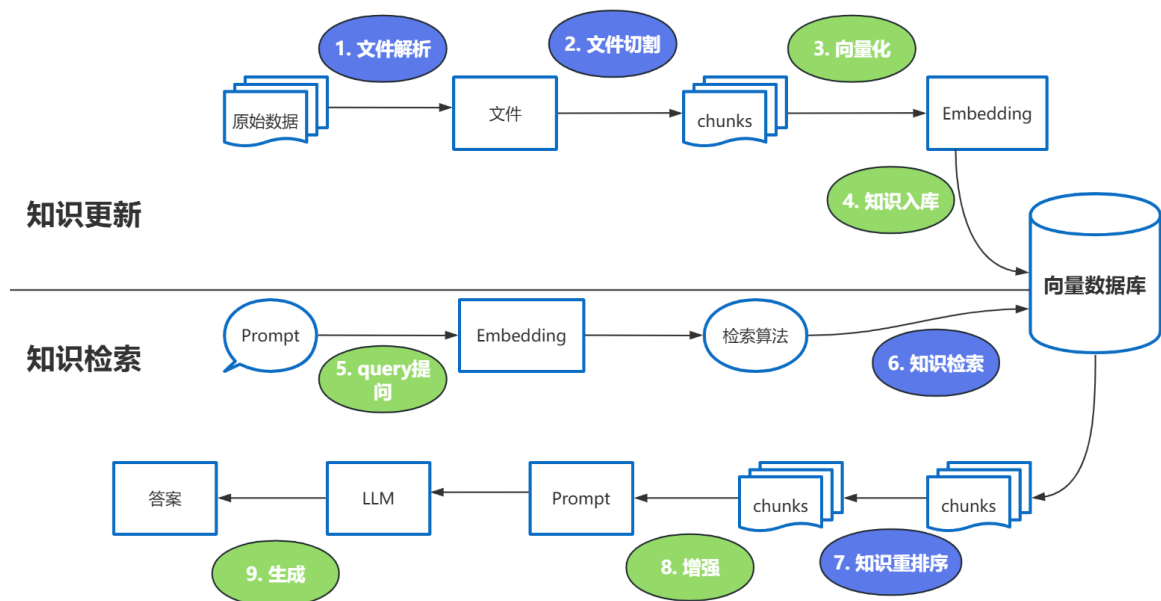
何为RAG？

Retrieval-Augmented Generation (检索增强生成)





检索-增强-生成过程：检索可以理解为第10步，增强理解为第13步（这里的提示词包含检索到的数据），生成为第15步。



这里三个位置涉及到大型模型的使用：

- 第3步向量化时，需要使用EmbeddingModels。
- 第7步重排序时，需要使用RerankModels。可对初步召回的较多 chunk（如 top 20 或 top 50）进行精排，提高召回准确率，防止LLM 处理无关信息，减少时间和成本。
- 第9步生成答案时，需要使用LLM。

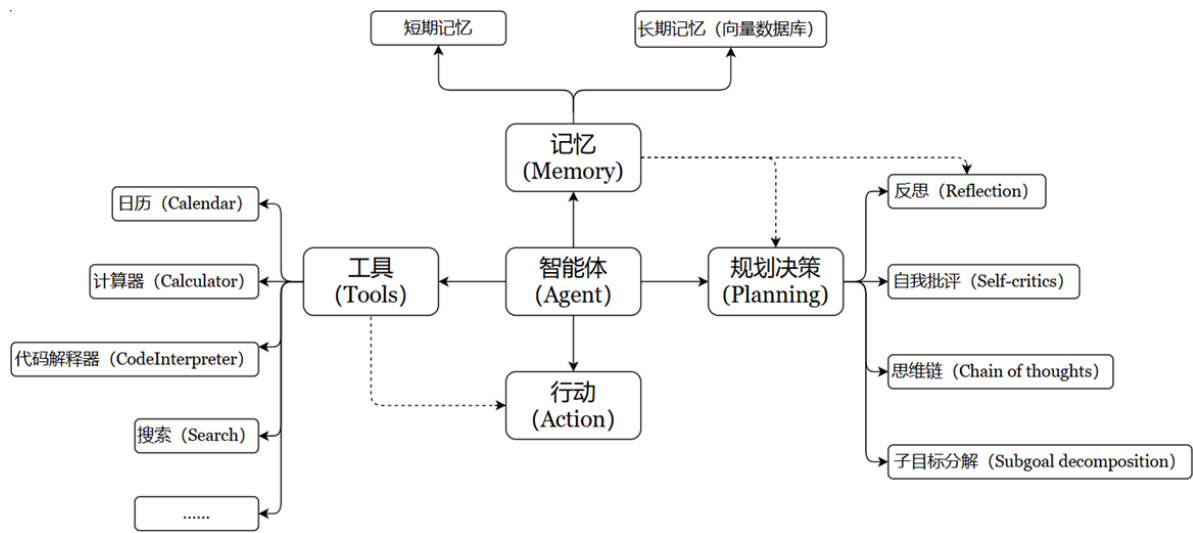
Reranker的使用场景：

- 适合：追求回答高精度和高相关性的场景中特别适合使用 Reranker，例如专业知识库或者客服系统等应用。
- 不适合：Reranker相较于RAG的成本更高。此外，引入reranker会增加召回时间，增加检索延迟。服务对响应时间要求高时，使用reranker可能不合适。

场景2：基于Agent架构的开发

充分利用 LLM 的推理决策能力，通过增加规划、记忆和工具调用的能力，构造一个能够独立思考、逐步完成给定目标的智能体。

OpenAI的元老翁丽莲于2023年6月在个人博客首次提出了现代AI Agent架构。



一个数学公式来表示：

Agent = LLM + Memory + Tools + Planning + Action

智能体核心要素被细化为以下模块：

1、大模型 (LLM) 作为 “大脑”： 提供推理、规划和知识理解能力，是AI Agent的决策中枢。

大脑主要由一个大型语言模型 LLM 组成，承担着信息处理和决策等功能， 并可以呈现推理和规划的过程，能很好地应对未知任务。

2、记忆 (Memory)

智能体像人类一样，能留存学到的知识以及交互习惯等，这样的机制能让智能体在处理重复工作时调用以前的经验，从而避免用户进行大量重复交互。

- **短期记忆：** 存储单次对话周期的上下文信息，属于临时信息存储机制。受限于模型的上下文窗口长度。



康师傅

02/23 23:12

帮我随机生成40行左右的数据，包括品牌、大区、销售额。其中品牌有安踏、耐克、李宁、阿迪、匹克、鸿星尔克，大区为中国的省份，销售额范围10000 - 100000。每个品牌在多个省份销售。

Tokens: 91



deepseek-ai/DeepSeek-V3 | 硅基流动

02/23 23:12

以下是随机生成的40行数据示例，包括品牌、大区（中国省份）和销售额：

品牌	大区	销售额 (元)
安踏	广东	85000
耐克	北京	92000



康师傅

02/23 23:15

上述数据没问题。但是需要按品牌分组，重新排列一下。

Tokens: 25



deepseek-ai/DeepSeek-V3 | 硅基流动

02/23 23:15

好的！以下是按照品牌分组重新排列后的数据示例：

按品牌分组数据

安踏

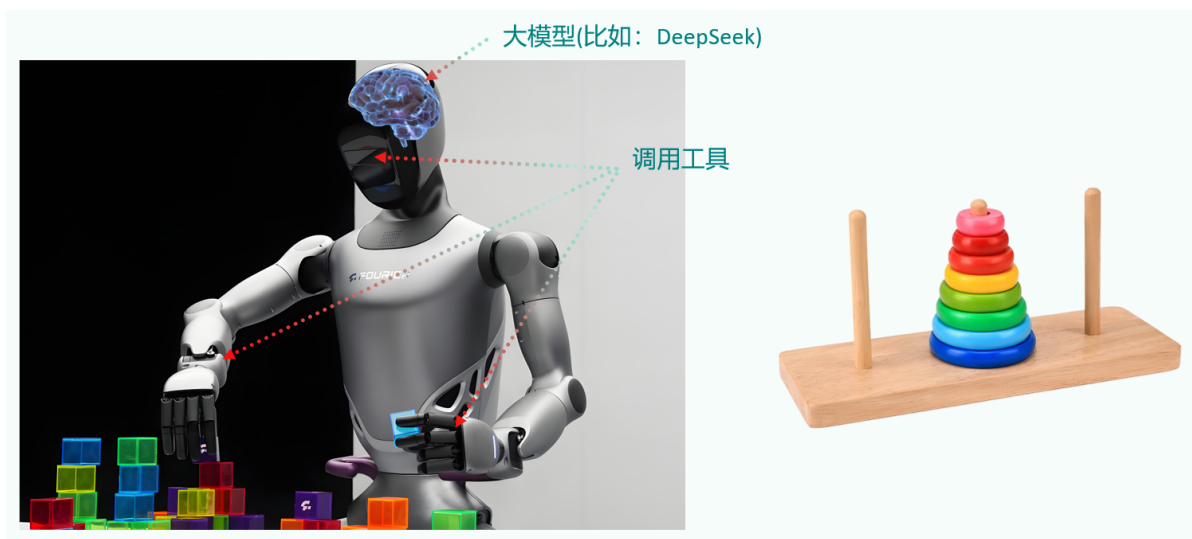
品牌	大区	销售额 (元)
安踏	广东	85000

- **长期记忆**：可以横跨多个任务或时间周期，可存储并调用核心知识，非即时任务。
 - 长期记忆，可以通过**模型参数微调（固化知识）**、**知识图谱（结构化语义网络）**或**向量数据库（相似性检索）**方式实现。

以人作类比：

- 短期记忆：在进行心算时临时记住几个数字
- 长期记忆：学会骑自行车后，多年后再次骑起来时仍能掌握这项技能

3、工具使用 (Tool Use) ：调用外部工具（如API、数据库）扩展能力边界。



4、规划决策 (Planning)：通过任务分解、反思与自省框架实现复杂任务处理。例如，利用思维链 (Chain of Thought) 将目标拆解为子任务，并通过反馈优化策略。

 秘塔AI搜索

没有广告，直达结果

对比QWQ和DeepSeek-r1|

☒ 全网 ☐ 长思考·R1



简洁

深入

 研究

对比QWQ和DeepSeek-r1

≡ 研究中...

步骤拆解 ▾

▼ QWQ的定义和背景

QWQ的定义和背景可以从多个角度进行解释：

▼ DeepSeek-r1的定义和背景

▼ ①中的主要功能和技术特点

▼ ②中的主要功能和技术特点

▼ ①中的应用场景和市场定位

▼ ②中的应用场景和市场定位

▼ ①中与②中的性能指标对比

▼ 用户对①中和②中的评价反馈

步骤

QWQ的定义和背景

DeepSeek-r1的定义和背景

①中的主要功能和技术特点

②中的主要功能和技术特点

①中的应用场景和市场定位

②中的应用场景和市场定位

①中与②中的性能指标对比

用户对①中和②中的评价反馈

最终回答

5、行动 (Action)： 实际执行决策的模块，涵盖软件接口操作（如自动订票）和物理交互（如机器人执行搬运）。比如：检索、推理、编程等。

智能体会形成完整的计划流程。例如先读取以前工作的经验和记忆，之后规划子目标并使用相应工具去处理问题，最后输出给用户并完成反思。

3、主线1：Dify平台私有化部署

3.1 Dify平台的介绍

Dify 作为一个综合性的 LLM 应用开发平台，内置了构建现代生成式 AI 应用所需的几乎所有关键技术栈。

它的具体功能如下：

- 基于Agent架构构建智能体应用
- 基于RAG构建私有知识库应用
- 基于Workflow构建智能工作流应用

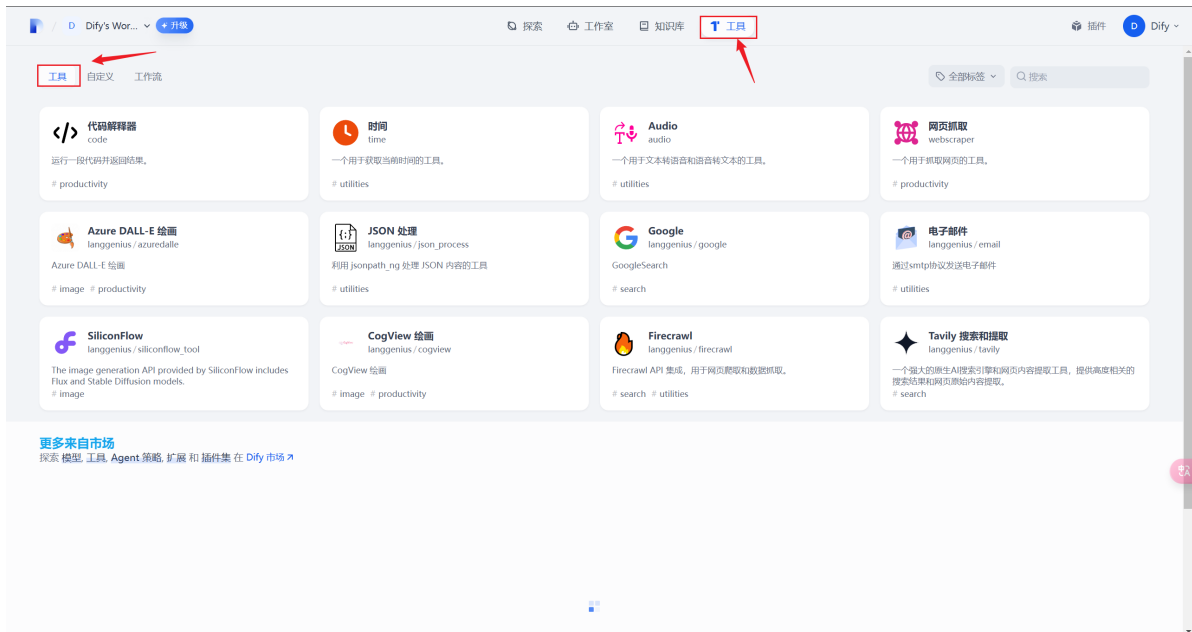
Dify 是当今最优雅、门槛最低、最受欢迎、效果最好的大模型开发平台之一。

无论是经验丰富的程序员还是初涉AI领域的团队（如产品经理、运营人员），都能够快速、高效地搭建并运营生产级别的生成式 AI 应用。

官网：<https://dify.ai/zh>

文档说明：https://github.com/langgenius/dify/blob/main/README_CN.md

说明：访问Dify官网需要魔法（或梯子、科学上网）



Dify 同时提供了一套易用的图形用户界面 (GUI) 和完善的 API 接口。这为开发者节省了大量重复"造轮子" (例如, 与不同模型 API 对接、管理对话状态、实现 RAG 流程等) 的时间, 使其可以将更多精力专注在业务逻辑的创新和满足特定用户需求上。

3.2 租赁Dify服务器：腾讯云

企业用户可以选择租用云服务器, 或者在本地的服务器中部署Dify。因为Dify所需的资源很小, 一个轻量级的服务器足以支持运行。

我们需要租赁一个云服务器去运行Dify服务：腾讯云。

官网: <https://cloud.tencent.com/>

① 基础配置

<https://buy.cloud.tencent.com/cvm>

如果是企业中使用或者个人资金充裕且业务稳定的话, 可以选择长租使用。期望优惠的话, 可以选择竞价实例。竞价实例, 在性能和稳定性上, 与按量计费模式没有差别。

竞价实例, 只要有人租长期的服务器就有可能把你的服务器踢掉, 实例被竞价释放也是有解决办法的, 后续会去讲。

腾讯云

选购腾讯云产品

10003905

控制台

云服务器 CVM

产品文档 产品价格 产品控制台

快速配置 自定义配置

1 选择基础配置

2 设置网络配置

3 确认配置信息

选择实例规格

创建专属资源组：创建4核8GB起至32元/月，尊享最低1.6折续费，即购即用《云资源包》《腾讯云服务器》

购买推荐

建议选购腾讯云最新一代超微服务器，性能强劲的云服务器SAG。

查看详情

依相关法律法规及监管要求，腾讯云上客户利用云服务从事虚拟货币相关业务，可查看详情。

基础配置

计费模式

包年包月

按量计费

我的实例

地域

中国

上海

南京

广州

北京

成都

重庆

中国香港

可用区

随机可用区

上海五区

上海八区

上海二区

实例配置

实例

架构

实例族

已选

SA9.MEDIUM2 (标准型SA9, 2核2GB)

数量

1

配置费用

¥0.09

下一步：设置网络配置

地域选择：没有要求，自己根据需要选即可。

实例配置：根据自己需求选择，无具体要求。这里我选择4核8GB。

上海

南京

广州

北京

成都

重庆

中国香港

处于不同地域的云产品内网不通，创建成功后不支持切换地域，建议选择最靠近您客户的地域，可降低访问延迟。

可用区

随机可用区

上海五区

上海八区

上海二区

相同地域下不同可用区的云产品内网可互通，已为您随机选择至上海五区

实例配置

实例

架构

实例族

已选实例

4核

8GB

全部架构

X86计算

ARM计算

异构计算

裸金属云服务器

高性能计算集群

全部实例族

标准型

内存型

计算型

GPU机型

NPU机型

大数据型

其他实例族

已选实例 S5.LARGE8 (标准型S5, 4核8GB)

已为您随机选择至上海五区，如需更多配额，可前往控制台申请，对随机可用区有疑问

开启机型对比

隐藏售罄产品

已选

S5.LARGE8 (标准型S5, 4核8GB)

时长

半年

7.3折

多买多折

数量

1

配置费用

¥585.00

新客专享

共省 ¥585.00

下一步：设置网络配置

镜像：选择CentOS、Ubuntu都可以，这里使用了Ubuntu。选择后点击下一步。

竞价出价模式

即随市场价格

镜像

公共镜像

自定义镜像

共享镜像

云镜像市场

TencentOS

OpenCloudOS

CentOS

Windows

Ubuntu

Debian

Ubuntu Server 24.04 LTS 64位

用途

类型

容量

数量

总性能

系统盘

通用型SSD云硬盘

50

GB

1

基础性能：2550IOPS, 110MB/s 带宽

添加数据盘

您还可以添加 20 块数据盘

已选

SA9.LARGE16 (标准型SA9, 4核16GB)

数量

1

配置费用

¥0.21

共省 ¥0.64

下一步：设置网络配置

⚠️ 竞价实例详情确认

竞价实例按小时收费，和按量计费模式相比拥有较大折扣，在性能和稳定性上，与按量计费模式没有差别。**当库存不足时，竞价实例可能会被自动释放，实例数据不会保留。**不可中断的单点服务、数据库等，应避免选择竞价实例，使用时请做好数据备份等保障工作。订阅“竞价实例回收通知”可前往 [消息中心](#)，更多竞价实例说明请 [查看文档](#)

[取消](#) [确认已知风险](#)

配置费用 **¥0.15**
共省 **¥0.41**

下一步：设置网络和主机

② 设置网络和主机

拉满带宽上限，新建安全组，把常用的端口都开启

网络与带宽

网络 ② [子网剩余可用IP 4092 个](#)

您的实例在上海五区，仅支持选择该可用区下的子网
当前网络为默认私有网络，建议您根据业务需要进行调整
如现有私有网络/子网不符合您的要求，可以去控制台 [新建私有网络](#) 或 [新建子网](#)。云主机购买后可以通过控制台切换私有网络完成私有网络/子网的切换

☐ 手动分配IP地址

公网 IP ② ☒ 分配独立公网 IP

线路类型

带宽计费模式 ②

注意：流量费用每小时结算一次，当客户余额不足时，两小时内将被停止流量服务。腾讯云默认可享受不超过2Gbps DDOS 防护，为腾讯云上公网 IP 秒级开启防护，减少因恶意攻击产生的流量费。

带宽上限 ② Mbps

IPv6 地址 所选的VPC/子网未开通IPv6 [去开通](#)

安全组

安全组 ②

放通常用IP端口 [如您有业务需要放通其他端口，您可以 \[新建安全组\]\(#\)](#)

☒ ICMP (公网 Ping 云服务器) ☒ TCP:22 (SSH 远程连接 Linux 实例) ☒ TCP:3389 (RDP 远程连接 Windows 实例)

☒ TCP:80 (云服务器作 Web 服务器 (HTTP)) ☒ TCP:443 (云服务器作 Web 服务器 (HTTPS)) ☒ 放通内网 (不同云资源间内网互通 (IPv4))

注意：来源目标为0.0.0.0/0 表示所有IP地址都可以用于访问，建议填写您常用的IP地址

请确保所选安全组已放通22端口(Linux SSH登录)，3389端口(Windows远程登录)，否则无法远程登录CVM，您可以进入CVM控制台 [修改安全组](#)

[查看安全组规则](#)

命名实例，设置密码，进行下一步

实例名称 ② 支持批量连续命名或指定模式命名，最多128个字符，您还可以输入113个字符

登录方式 ②

登录名

密码 检测到您设置的密码强度较弱，可能有被入侵的风险，建议您设置12位及以上，至少包括4项：(a-z)(A-Z)(0-9)(!@#%*^&*+~_-<>?等字符)，每种字符大于等于2位且不能相同的密码。

确认密码

注意：请牢记您所设置的密码，如遗忘可登录CVM控制台重置密码。自定义引导的实例不支持保存为自动模板。

安全加固 ☒ 免费开通 安装组件免费开通DOS防护和主机安全基础版 ②

云监控 ☒ 免费开通 免费开通云产品监控、分析和实时告警，安装组件获取主机监控指标 ②

自动化助手 ☒ 免费开通 安装组件免费开通自动化助手，免密码、免SSH登录即可批量管理实例，执行命令，完成日常管理任务 ②

已选 SA9.LARGE16 (标准型SA9, 4核16GB) 数量

配置费用 **¥0.21**
公网流量费用 **¥0.80/GB**
共省 **¥0.64**

[上一步](#) [下一步：确认配置信息](#)

开通

云服务器 CVM

产品文档

产品价格

产品控制台

快速配置

自定义配置

选择基础配置

设置网络和主机

确认配置信息

基础和实例配置

主机计费模式

竞价实例

地域

上海

可用区

上海五区

实例

SA9.LARGE16 (标准型SA9, 4核16GB)

竞价出价模式

跟随市场价格

镜像

公共镜像 | Ubuntu | img-mmytdhbn | Ubuntu Server 24.04 LTS 64位 | 20GB

系统盘

通用型SSD云硬盘 | 50GiB

数据盘

未设置

网络和安全性

所属网络

vpc-ckpoph7y | Default-VPC (默认) | 172.17.0.0/16

所在子网

subnet-3klujzcr | Default-Subnet (默认) | 172.17.0.0/20

内网 IP

未分配

公网IP

购买

带宽计费模式

按流量计费100Mbps

线路类型

BGP

IPv6

未分配

安全组

自定义模板

其他设置

设置密码

保存为启动模板

生成 API Explorer 最佳实践脚本

协议

☒ 我已阅读并同意《腾讯云服务协议》、《腾讯云禁止虚拟货币相关活动声明》

风险确认

☒ 我已知晓将竞价实例可能会自动释放，并评估好业务需求承担的风险更多说明

已选 SA9.LARGE16 (标准型SA9, 4核16GB)

数量

-

1

+

配置费用 ¥0.21
共选 ¥0.64

公网流量费用 ¥0.80/GB

上一步

开通

③ 登录(使用Xshell或finalshell)

登录

×

腾讯云产品

云服务器 (CVM)

ins-52bm5i4s - IP: 146.56.198.254

连接协议

☐ 免密连接 (TAT)

☒ 终端连接 (SSH)

连接网络

连接端口

公网 146.56.198.254

22

验证方式

☒ 密码验证

☐ 密钥验证

输入密码

使用托管密码

用户名

密码

ubuntu

请输入密码

忘记密码?

☒ 保存登录信息与登录凭证，下次快速登录

如何快速登录 >

登录

其他登录方式

VNC登录

?

创建好了，通过这个公网IP，端口使用22，账号ubuntu，密码使用你设置的密码。使用你的远程连接工具XShell 或 final shell 连接即可。

XShell界面如下：

```
* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro

System information as of Tue Jul 15 03:13:39 PM CST 2025

System load:  0.0                Processes:            145
Usage of /:   10.0% of 49.10GB   Users logged in:     1
Memory usage: 4%                IPv4 address for eth0: 10.206.0.7
Swap usage:   0%

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment.

  https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

248 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

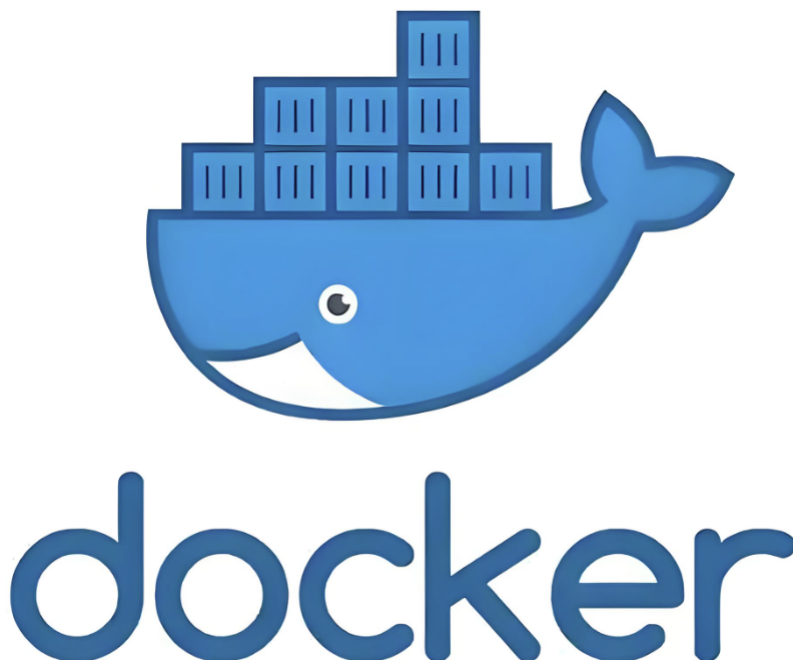
Last login: Tue Jul 15 15:12:03 2025 from 106.55.203.141
/usr/bin/xauth:  file /home/ubuntu/.Xauthority does not exist
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@VM-0-7-ubuntu:~$
```

3.3 部署Docker

部署dify平台，需要基于docker环境，而腾讯云新建的云平台上默认是没有docker的。接着，需要在腾讯云租用的服务器中部署Docker。

什么是Docker?



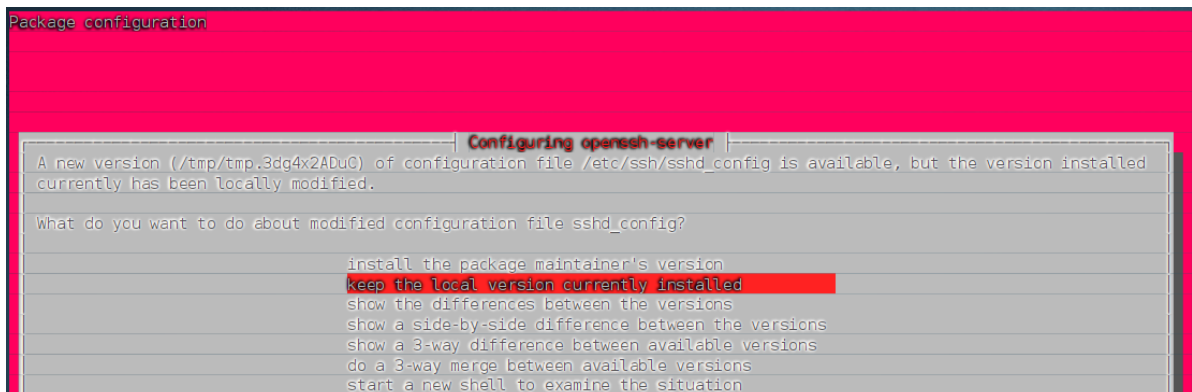
Docker是一种容器化技术，相较于传统的通过虚拟机技术实现的虚拟化方案来说，Docker是一种更加轻量级的虚拟化解决方案。

它可以将应用程序及其依赖项打包成一个独立的容器，并在不同的环境中运行。通过Docker容器，开发者可以轻松地构建、部署和运行应用程序，而无需担心环境配置和依赖问题。

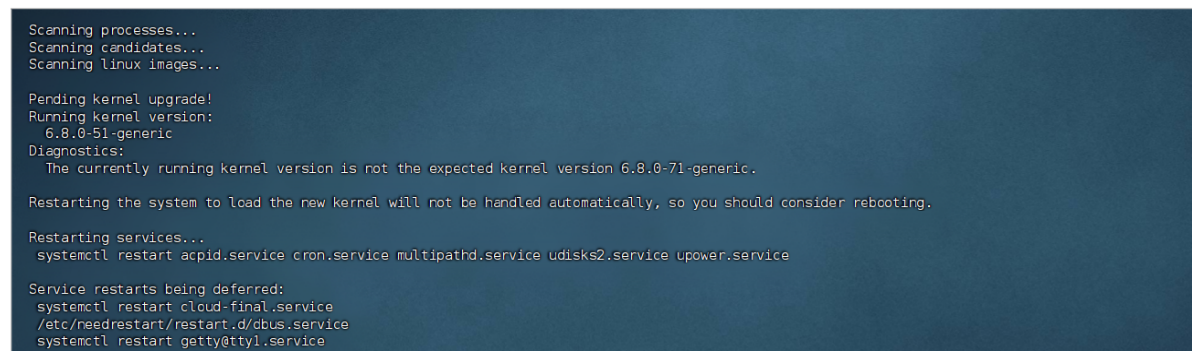
按照下面的指令一步一步进行操作

```
1  #更新软件包
2  sudo apt update
3
4  sudo apt upgrade
5
6  #安装docker依赖
7  sudo apt install software-properties-common
8
9  sudo apt-get install ca-certificates curl gnupg lsb-releasesudo
10 sudo apt-get install ca-certificates curl gnupg lsb-release
11
12 #添加Docker官方GPG密钥
13 curl -fsSL http://mirrors.aliyun.com/docker-ce/linux/ubuntu/gpg | sudo apt-key add -
14
15 #添加Docker软件源（输入后根据提示按Enter）
16 sudo add-apt-repository "deb [arch=amd64] http://mirrors.aliyun.com/docker-ce/linux/ubuntu
$(lsb_release -cs) stable"
17
18 #安装docker（输入后根据提示输入y）
19 sudo apt-get install docker-ce docker-ce-cli containerd.io
```

执行 `sudo apt upgrade` 的时候会出现这个界面，按回车即可



之后如果在这个界面卡住，按几下回车即可



安装完毕，启动docker，并查看状态

```
1 sudo systemctl start docker
2
3 sudo systemctl status docker
```

如图所示即为启动成功

```
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Tue 2025-07-15 15:45:41 CST; 30s ago
     TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
    Main PID: 35932 (dockerd)
      Tasks: 10
     Memory: 22.8M (peak: 23.6M)
        CPU: 150ms
    CGroup: /system.slice/docker.service
            └─35932 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Jul 15 15:45:40 VM-0-7-ubuntu dockerd[35932]: time="2025-07-15T15:45:40.856634568+08:00" level=info msg=>
Jul 15 15:45:40 VM-0-7-ubuntu dockerd[35932]: time="2025-07-15T15:45:40.873933788+08:00" level=info msg=>
Jul 15 15:45:40 VM-0-7-ubuntu dockerd[35932]: time="2025-07-15T15:45:40.894854680+08:00" level=info msg=>
Jul 15 15:45:41 VM-0-7-ubuntu dockerd[35932]: time="2025-07-15T15:45:41.031594183+08:00" level=info msg=>
Jul 15 15:45:41 VM-0-7-ubuntu dockerd[35932]: time="2025-07-15T15:45:41.045137763+08:00" level=info msg=>
Jul 15 15:45:41 VM-0-7-ubuntu dockerd[35932]: time="2025-07-15T15:45:41.045202768+08:00" level=info msg=>
Jul 15 15:45:41 VM-0-7-ubuntu dockerd[35932]: time="2025-07-15T15:45:41.059603679+08:00" level=info msg=>
Jul 15 15:45:41 VM-0-7-ubuntu dockerd[35932]: time="2025-07-15T15:45:41.063223879+08:00" level=info msg=>
Jul 15 15:45:41 VM-0-7-ubuntu dockerd[35932]: time="2025-07-15T15:45:41.063272006+08:00" level=info msg=>
Jul 15 15:45:41 VM-0-7-ubuntu systemd[1]: Started docker.service - Docker Application Container Engine.
~
lines 1-22/22 (END)
```

看到running状态说明docker已经正常启动

3.4 部署Dify

官网: <https://github.com/langgenius/dify>

这里我们可以参考Github官网的几个步骤

Quick start

Before installing Dify, make sure your machine meets the following minimum system requirements:

- CPU $\geq$ 2 Core
- RAM $\geq$ 4 GiB

The easiest way to start the Dify server is through [docker compose](#). Before running Dify with the following commands, make sure that [Docker](#) and [Docker Compose](#) are installed on your machine:

```
cd dify
cd docker
cp .env.example .env
docker compose up -d
```

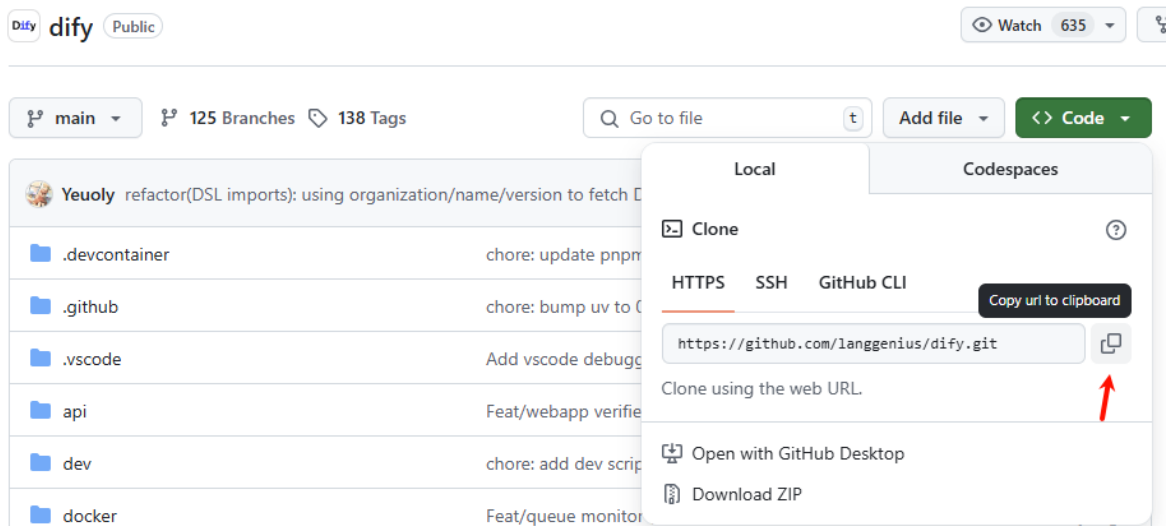
① 下载源码

在 `/opt` 下创建一个dify目录

```
1 cd /opt
2 sudo mkdir dify
```

```
ubuntu@VM-0-7-ubuntu:~$ cd /opt
ubuntu@VM-0-7-ubuntu:/opt$ sudo mkdir dify
ubuntu@VM-0-7-ubuntu:/opt$ ll
total 16
drwxr-xr-x  4 root root 4096 Jul 15 15:49 ./
drwxr-xr-x 24 root root 4096 Jul 15 15:49 ../
drwx--x--x  4 root root 4096 Jul 15 15:45 containerd/
drwxr-xr-x  2 root root 4096 Jul 15 15:49 dify/
ubuntu@VM-0-7-ubuntu:/opt$ █
```

然后在其Github主页进行复制



在 `/opt` 目录下执行 `git clone` 命令，这里我们可以通过镜像站进行下载

```
1 git clone https://github.com/langgenius/dify.git
```

如果提示权限不足，可以使用如下指令，临时提权

```
1 sudo git clone https://github.com/langgenius/dify.git
```

如图即为下载完成

```
ubuntu@VM-0-7-ubuntu:/opt$ sudo git clone https://github.com/langgenius/dify.git
Cloning into 'dify'...
remote: Enumerating objects: 186111, done.
remote: Counting objects: 100% (107/107), done.
remote: Compressing objects: 100% (65/65), done.
remote: Total 186111 (delta 73), reused 42 (delta 42), pack-reused 186004 (from 3)
Receiving objects: 100% (186111/186111), 100.79 MiB | 49.00 KiB/s, done.
Resolving deltas: 100% (136960/136960), done.
ubuntu@VM-0-7-ubuntu:/opt$ ll
total 16
drwxr-xr-x  4 root root 4096 Jul 15 16:06 ./
drwxr-xr-x 24 root root 4096 Jul 15 17:28 ../
drwx--x--x  4 root root 4096 Jul 15 15:45 containerd/
drwxr-xr-x 13 root root 4096 Jul 15 17:27 dify/
```

注意，如果使用github下载过慢，还可以使用码云（Gitee）或镜像网站替代 GitHub 直接下载，利用国内服务器加速。

操作步骤：

- 1) 注册码云账号 (<https://gitee.com>) 。
- 2) 在码云新建仓库，选择「导入GitHub仓库」，粘贴 <https://github.com/langgenius/dify.git> 的链接。
- 3) 导入完成后，使用码云生成的仓库地址克隆：

```
1 sudo git clone https://gitee.com/你的用户名/dify.git
```

这里大家也可以直接使用我的链接：

```
1 sudo git clone https://gitee.com/shkstart/dify.git
```

```
ubuntu@VM-0-10-ubuntu:/opt$ sudo git clone https://gitee.com/shkstart/dify.git
Cloning into 'dify'...
remote: Enumerating objects: 187550, done.
remote: Counting objects: 100% (187550/187550), done.
remote: Compressing objects: 100% (45795/45795), done.
remote: Total 187550 (delta 138201), reused 187550 (delta 138201), pack-reused 0 (from 0)
Receiving objects: 100% (187550/187550), 101.23 MiB | 8.09 MiB/s, done.
Resolving deltas: 100% (138201/138201), done.
ubuntu@VM-0-10-ubuntu:/opt$
```

② 使用docker启动Dify

首先进入dify路径中的docker文件夹中

```
ubuntu@VM-0-10-ubuntu:/$ cd /opt/dify/docker
ubuntu@VM-0-10-ubuntu:/opt/dify/docker$ ll
total 224
drwxr-xr-x  9 root root  4096 Jul 16 23:12 ./
drwxr-xr-x 12 root root  4096 Jul 16 23:12 ../
drwxr-xr-x  2 root root  4096 Jul 16 23:12 certbot/
drwxr-xr-x  2 root root  4096 Jul 16 23:12 couchbase-server/
-rw-r--r--  1 root root  5057 Jul 16 23:12 docker-compose.middleware.yaml
-rw-r--r--  1 root root 63694 Jul 16 23:12 docker-compose.png
-rw-r--r--  1 root root 20394 Jul 16 23:12 docker-compose-template.yaml
-rw-r--r--  1 root root 42492 Jul 16 23:12 docker-compose.yaml
drwxr-xr-x  2 root root  4096 Jul 16 23:12 elasticsearch/
-rw-r--r--  1 root root 31118 Jul 16 23:12 .env.example
-rwxr-xr-x  1 root root  4097 Jul 16 23:12 generate_docker_compose*
-rw-r--r--  1 root root  3057 Jul 16 23:12 middleware.env.example
drwxr-xr-x  4 root root  4096 Jul 16 23:12 nginx/
-rw-r--r--  1 root root  6605 Jul 16 23:12 README.md
drwxr-xr-x  2 root root  4096 Jul 16 23:12 ssrf_proxy/
drwxr-xr-x  2 root root  4096 Jul 16 23:12 startupscripts/
drwxr-xr-x  6 root root  4096 Jul 16 23:12 volumes/
ubuntu@VM-0-10-ubuntu:/opt/dify/docker$
```

```
1 sudo docker compose up -d
```

执行失败，大概率会由于网络问题或镜像缺失问题发生报错。

```
! weaviate      Interrupted
! db            Interrupted
x api Error     Get "https://registry-1.docker.io/v2/": read tcp 10.206.0.7:43190->103.246.246.144:443: read: connection reset by peer
! worker       Interrupted
! sandbox      Interrupted
! plugin_daemon Interrupted
! web          Interrupted
! redis        Interrupted
! nginx        Interrupted
! ssrf_proxy   Interrupted
Error response from daemon: Get "https://registry-1.docker.io/v2/": read tcp 10.206.0.7:43190->103.246.246.144:443: read: connection reset by peer
```

```
(py310) root@vmInstance6b5zmhdo:/data/dify/dify/docker# docker compose up -d
[+] Running 10/10
! db Interrupted
! ssrf_proxy Interrupted
! api Interrupted
! sandbox Interrupted
! nginx Interrupted
X plugin_daemon Error Get "https://registry-1.docker.io/v2/": net/http: request canceled while waiting for connection (Client.Timeout exceeded while awaiting headers)
! weaviate Interrupted
! worker Interrupted
! redis Interrupted
! web Interrupted
Error response from daemon: Get "https://registry-1.docker.io/v2/": net/http: request canceled while waiting for connection (Client.Timeout exceeded while awaiting headers)
(py310) root@vmInstance6b5zmhdo:/data/dify/dify/docker#
```

进行镜像源的配置

```
1 sudo vi /etc/docker/daemon.json
```

添加下面的配置

```
1 {
2   "registry-mirrors": [
3     "https://docker.unsee.tech",
4     "https://dockerpull.org",
5     "https://docker.1panel.live",
6     "https://dockerhub.icu",
7     "https://docker.m.daocloud.io",
8     "https://docker.nju.edu.cn",
9     "https://registry.docker-cn.com",
10    "https://docker.mirrors.ustc.edu.cn",
11    "https://hub-mirror.c.163.com",
12    "https://mirror.baidubce.com",
13    "https://5tqw56kt.mirror.aliyuncs.com",
14    "https://docker.hpcloud.cloud",
15    "http://mirrors.ustc.edu.cn",
16    "https://docker.chenby.cn",
17    "https://docker.ckyl.me",
18    "http://mirror.azure.cn",
19    "https://hub.rat.dev"
20  ]
21 }
```

```

{
    "registry-mirrors": [
        "https://docker.unsee.tech",
        "https://dockerpull.org",
        "https://docker.lpanel.live",
        "https://dockerhub.icu",
        "https://docker.m.daocloud.io",
        "https://docker.nju.edu.cn",
        "https://registry.docker-cn.com",
        "https://docker.mirrors.ustc.edu.cn",
        "https://hub-mirror.c.163.com",
        "https://mirror.baidubce.com",
        "https://5tqw56kt.mirror.aliyuncs.com",
        "https://docker.hpcloud.cloud",
        "http://mirrors.ustc.edu.cn",
        "https://docker.chenby.cn",
        "https://docker.ckyl.me",
        "http://mirror.azure.cn",
        "https://hub.rat.dev"
    ]
}
~
~
~
~
"/etc/docker/daemon.json" 21L, 634B

```

保存，然后在终端重新启动一下docker

```

1  # 重新登陆，需要输入密码
2  systemctl daemon-reload
3
4  systemctl restart docker

```

重新执行

```

1  sudo docker compose up -d

```

开始正常下载了

```

(py310) root@vmInstance6b5zmdo:/data/dify/dify/docker# docker compose up -d
[+] Running 10/81
  ⚙️ weaviate [    ] Pulling
  ⚙️ web [      ] Pulling
  ✓ redis Pulled
  ⚙️ api [      ] Pulling
  ⚙️ plugin_daemon [  ⚙️ ] Pulling
  ⚙️ ssrf_proxy [   ] Pulling
  ⚙️ nginx [     ] Pulling
  ⚙️ sandbox [    ] Pulling
  ⚙️ db [       ] Pulling
  ⚙️ worker Pulling

```

注意：可能出现报错，报错如下

```
(py310) root@vmInstance6b5zmhdo:/data/dify/dify/docker# docker compose up -d
[*] Running 0/6
Container docker-web-1 Starting 0.0s
Container docker-redis-1 Starting 0.0s
Container docker-weaviate-1 Starting 0.0s
Container docker-ssrf_proxy-1 Starting 0.0s
Container docker-db-1 Starting 0.0s
Container docker-sandbox-1 Starting 0.0s
Error response from daemon: AppArmor enabled on system but the docker-default profile could not be loaded: running '/usr/sbin/apparmor_parser -Kr /var/lib/docker/tmp/docker-default3989112974' failed with output: AppArmor parser error for /var/lib/docker/tmp/dock
er-default3989112974 in profile /etc/apparmor.d/tunables/home.d/ubuntu at line 7: Lesser found unexpected character: ' (0a0) in state: INITIAL
```

于是根据报错信息检查

```
1 sudo vi /etc/apparmor.d/tunables/home.d/ubuntu
```

删除掉报错信息中第七行的多余字符即可

```
# This file is auto-generated. It is recommended you update it using:
# $ sudo dpkg-reconfigure apparmor
#
# The following is a space-separated list of where additional user home
# directories are stored, each must have a trailing '/'. Directories added
# here are appended to @{HOMEDIRS}. See tunables/home for details.
~
~
~
```

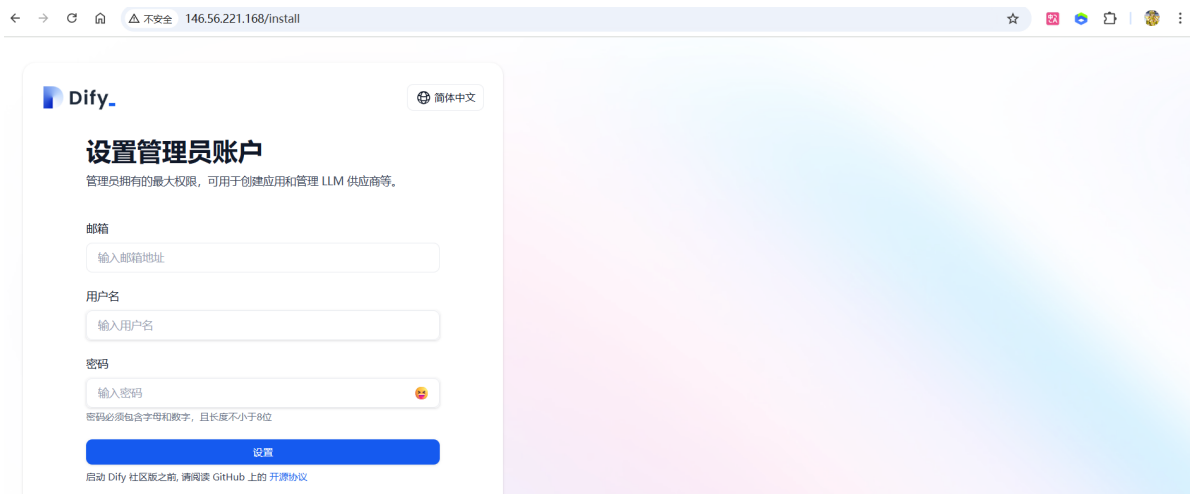
重新运行，成功

```
(py310) root@vmInstance6b5zmhdo:/data/dify/dify/docker# docker compose up -d
[+] Running 10/10
✔ Container docker-redis-1 Started
✔ Container docker-sandbox-1 Started
✔ Container docker-web-1 Started
✔ Container docker-weaviate-1 Started
✔ Container docker-ssrf_proxy-1 Started
✔ Container docker-db-1 Healthy
✔ Container docker-api-1 Started
✔ Container docker-worker-1 Started
✔ Container docker-nginx-1 Started
✔ Container docker-plugin_daemon-1 Started
```

③ 访问

根据自己的服务器进行地址的选择。比如说这里我的就是<http://146.56.221.168:80>

如图所示为成功访问，进行注册登陆即可



设置管理员账户

管理员拥有的最大权限，可用于创建应用和管理 LLM 供应商等。

邮箱

shkstart@126.com

用户名

shkstart

密码

.....



密码必须包含字母和数字，且长度不小于8位

设置

启动 Dify 社区版之前, 请阅读 [GitHub 上的 开源协议](#)

嗨，近来可好

👋 欢迎来到 Dify, 登录以继续

邮箱

shkstart@126.com

密码

[忘记密码?](#)

.....



登录

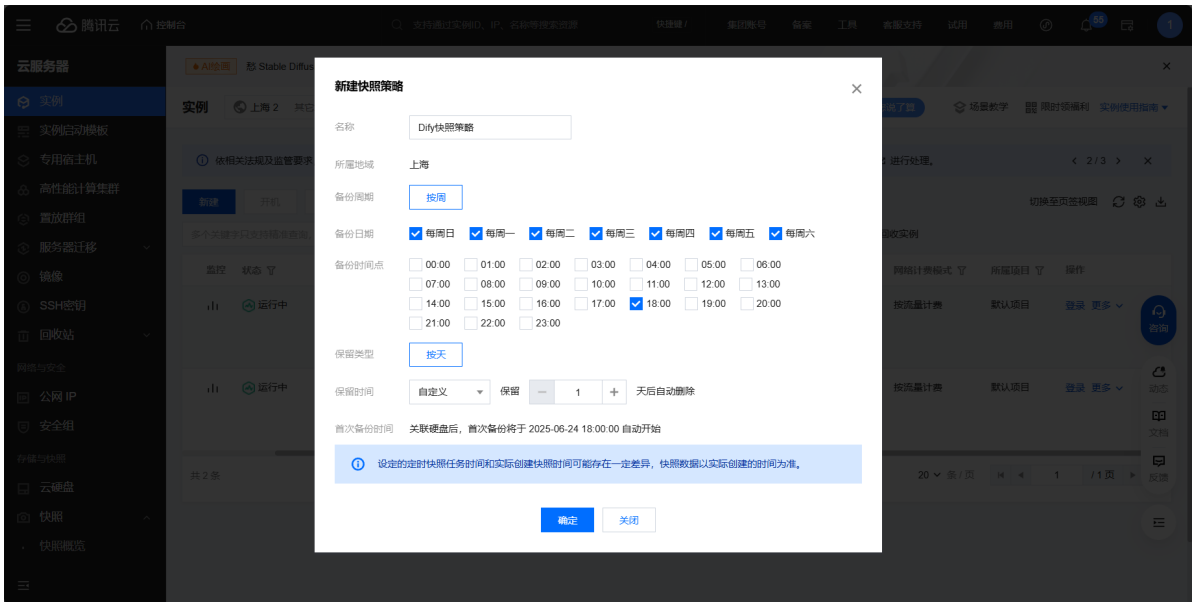
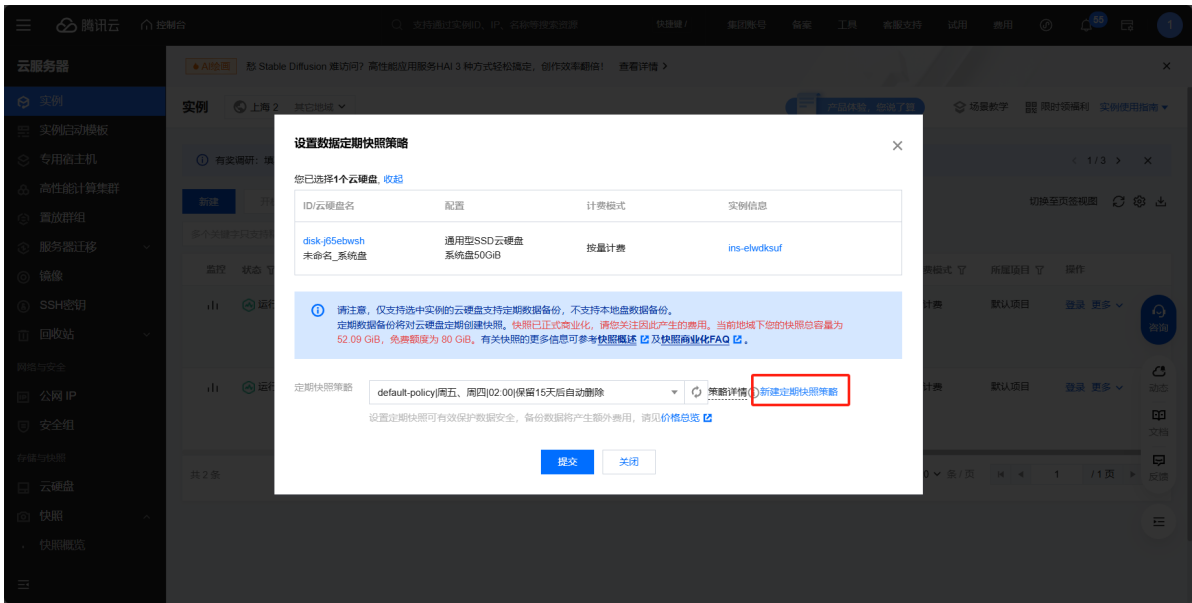
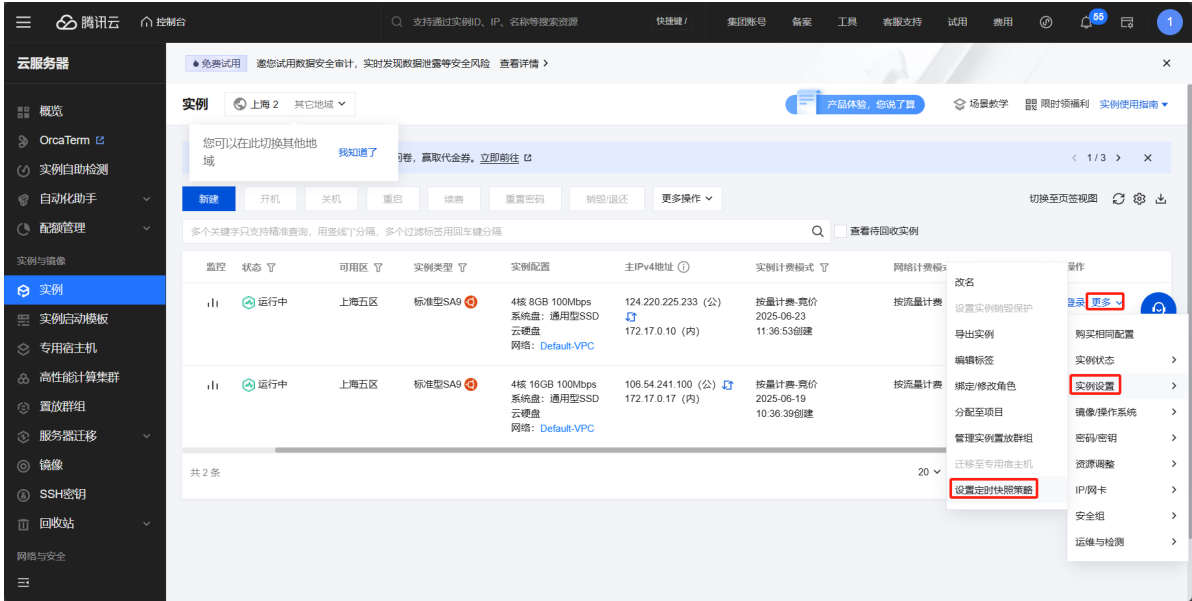
使用即代表您同意我们的 [使用协议](#) & [隐私政策](#)

如果您还没有初始化账户，请前往初始化页面 [设置管理员账户](#)

注意：如果一直无法加载进去，则需要重启docker再次尝试

④ 设置镜像

为避免案例中的竞价实例被释放，可以在控制台中的快照中设置快照策略，即使被释放了也能保存快照，从而快速恢复



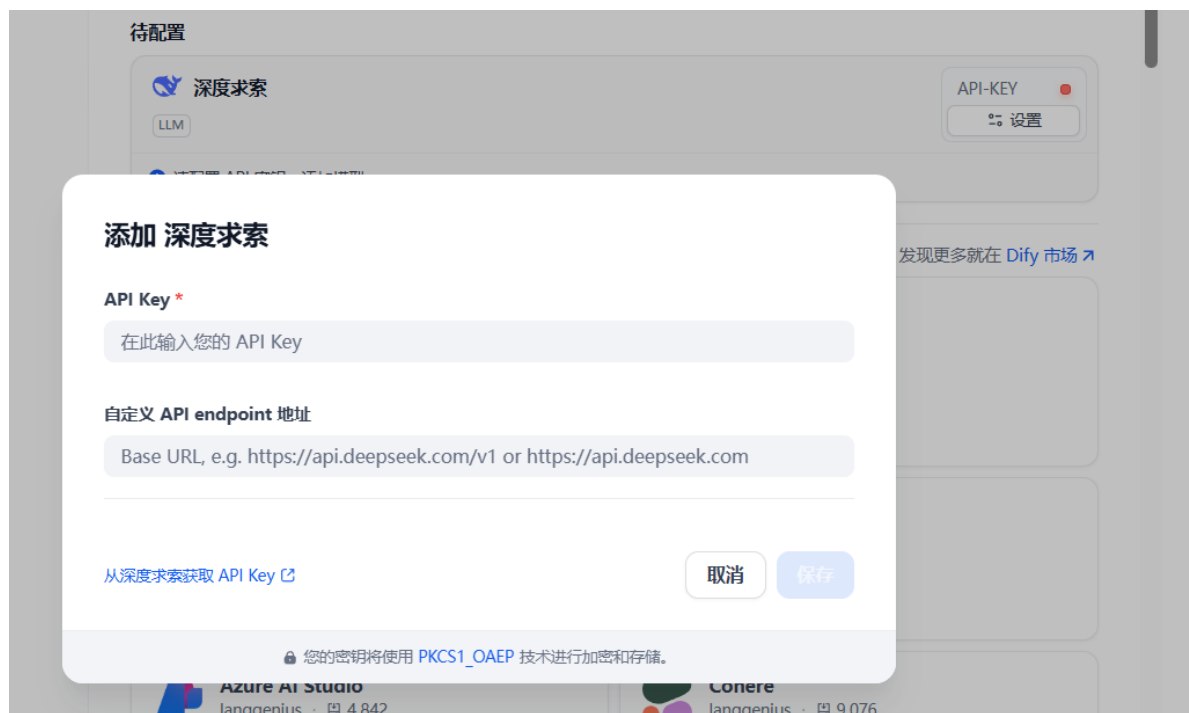
再次进入定期快照策略可发现已设置成功

3.5 部署在线大模型

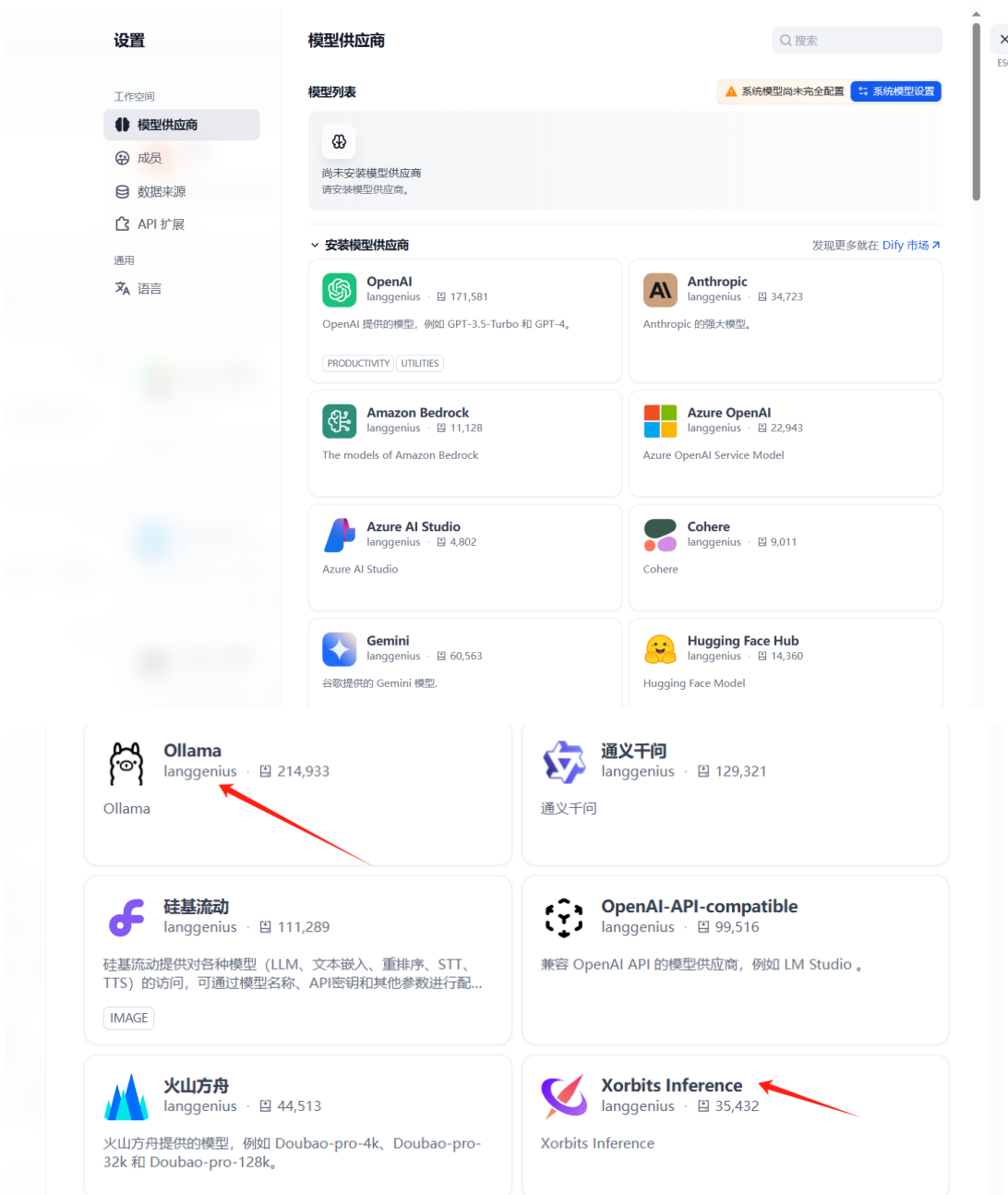
如果想调用线上的LLM，则可以用Dify选择线上的模型运营商。比如说可以在模型运营商中选择Deepseek



DeepSeek官网地址：<https://www.deepseek.com/>，在官网获取自己的API即可配置后使用



在这里我们可以使用提供的在线大模型运营商，但是为了考虑到可能存在的数据安全问题，所以我们自己部署Ollama和Xinference，进而部署私有的大模型。



4、主线2：大语言模型的部署

4.1 租赁GPU服务器：AutoDL

AutoDL介绍

这里我们选用AutoDL平台租赁服务器。这是一款面向开发者和企业的云计算平台，主要提供高性价比的 **GPU算力资源**，支持AIGC、深度学习、云游戏、渲染测绘、元宇宙、HPC等应用。

平台地址：<https://www.autodl.com/>

AutoDL服务器的资源比较紧俏，且比较贵

- 一台机器开机一个小时平均花费2元
- **建议**：一般早上开始工作的时候开机，在结束一天工作的时候关机。

① 配置服务器+镜像

选择服务器：

这里可以选择西北B区的单卡4090作为我们的服务器，我们需要租赁两台服务器，一台部署Ollama，另一台部署Xinference。

注意：在AutoDL平台上，只提供了6006端口进行开放。

The screenshot shows the AutoDL platform's server configuration interface. At the top, there's a navigation bar with '算力市场', 'AI服务器', '算法社区', '私有云', '帮助文档', and '更多'. Below this, a banner mentions a promotion for RTX 5090. The main area is titled '容器实例 / 创建实例' and includes a warning about WebUI usage. A sidebar on the right offers a '优惠券' (coupon) and a '控制台中' (in console) link. The configuration section includes:

- 计费方式** (Billing Method): '按量计费' (Pay-as-you-go) is selected.
- 选择地区** (Select Region): '北京B区' (Beijing B Zone) is selected, with '3090' and '4090' tags.
- GPU型号** (GPU Model): 'RTX 4090 (126/2288)' is selected.
- GPU数量** (GPU Count): '1' is selected.
- 选择主机** (Select Host): A table lists available hosts with columns for '主机ID', '算力型号/显存', '空间GPU', '每GPU分配', 'CPU型号', '硬盘', '驱动CUDA', and '价格(单卡)'. Hosts 053机, 039机, and 037机 are listed, all with RTX 4090 24GB and a price of ¥2.08/时.

At the bottom, the '日常费用' (Daily Fee) is ¥0.00/日, and the '配置费用' (Configuration Fee) is ¥2.08/时. A '创建并开机' (Create and Start) button is highlighted in blue.

选择镜像版本：

The screenshot shows the '选择镜像版本' (Select Image Version) section of the AutoDL platform. It includes:

- 镜像** (Image): '基础镜像' (Base Image) is selected, with '社区镜像' (Community Image) and '我的镜像' (My Image) as alternatives.
- 基础镜像包含常用基本软件** (Base Image includes commonly used basic software): A note mentions '深度学习框架、Miniconda等' (Deep learning frameworks, Miniconda, etc.).
- 镜像选择** (Image Selection): A dropdown menu shows 'PyTorch / 2.1.2 / 3.10(ubuntu22.04) / 11.8' as the selected option.
- 优惠券** (Coupon): A '请选择' (Please select) dropdown is shown.

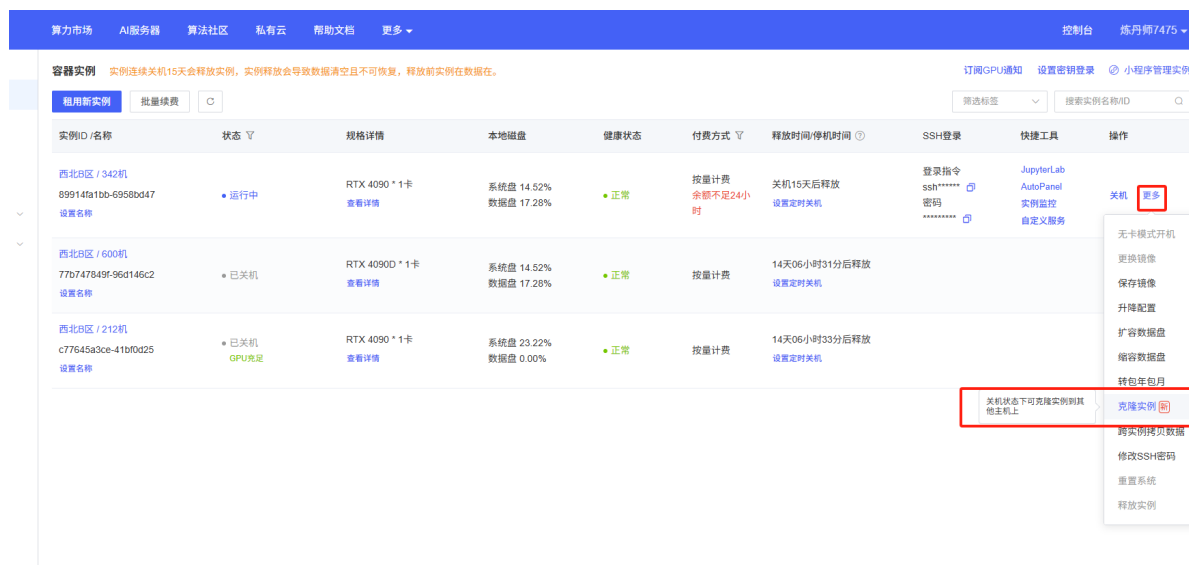
At the bottom, the '日常费用' (Daily Fee) is ¥0.00/日, and the '配置费用' (Configuration Fee) is ¥2.08/时. A '创建并开机' (Create and Start) button is highlighted in blue.

② XShell连接登录

复制该服务器的登录指令，通过远程连接工具进行登录

③ 克隆实例

在关机状态下，进行实例克隆，创建一个副本。



实例ID / 名称	状态	规格详情	本地磁盘	健康状态	付费方式	释放时间/停机时间	SSH登录	快捷工具	操作
西北B区 / 342机 89914fa1bb-6958bd47 设置名称	运行中	RTX 4090 * 1卡 查看详情	系统盘 14.52% 数据盘 17.28%	正常	按量计费 余额不足24小时	关机15天后释放 设置定时关机	登录指令 ssh***** 密码 *****	JupyterLab AutoPanel 实例监控 自定义服务	关机 更多
西北B区 / 600机 77b747849f-96d146c2 设置名称	已关机	RTX 4090D * 1卡 查看详情	系统盘 14.52% 数据盘 17.28%	正常	按量计费	14天06小时31分后释放 设置定时关机			无卡模式开机 更换镜像 保存镜像 升降配置 扩容数据盘 缩容数据盘 转包年包月 克隆实例
西北B区 / 212机 c77645a3ce-41bf0d25 设置名称	已关机 GPU充足	RTX 4090 * 1卡 查看详情	系统盘 23.22% 数据盘 0.00%	正常	按量计费	14天06小时33分后释放 设置定时关机			跨实例拷贝数据 修改SSH密码 重置系统 释放实例

设置一下备注名称

西北B区 / 342机

89914fa1bb-6958bd47

Ollama

西北B区 / 600机

77b747849f-96d146c2

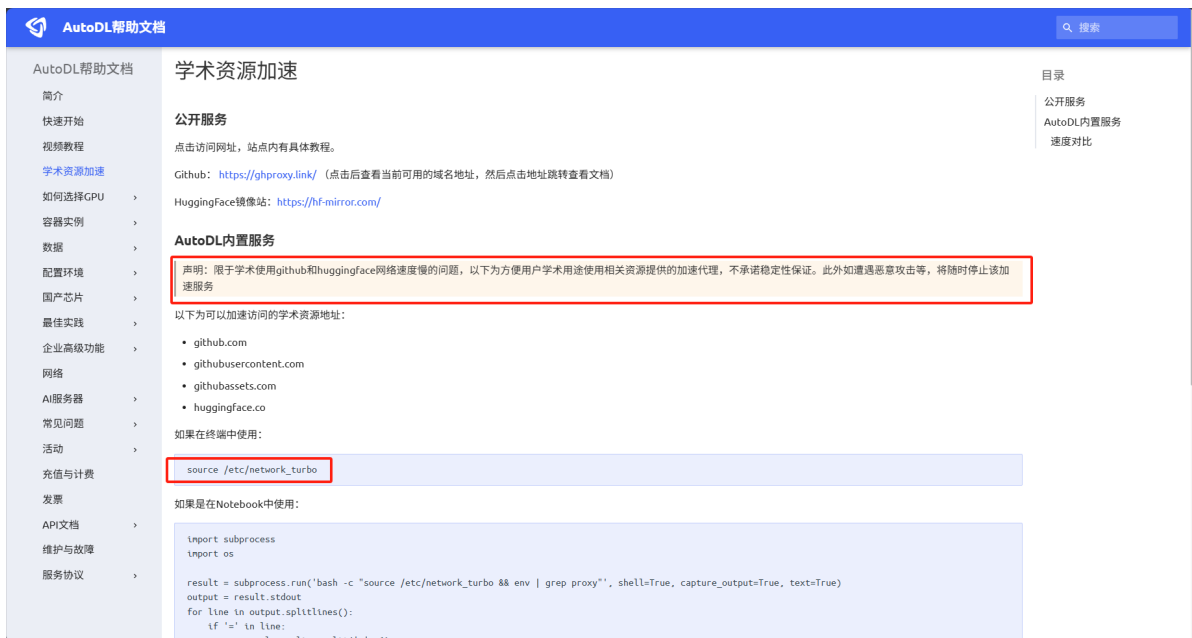
Xinference

④ 开启学术资源加速

为下载一些外网的资源（比如Github、HuggingFace等），需要在当前终端中开启 学术资源加速

免不了我们要在这个系统上安装一些软件。这些软件可能来自于如下的红框的位置。默认是下载不了的。那么就需要魔法或科学上网。这里我们称为：学术加速。

https://www.autodl.com/docs/network_turbo/



将框选住的一行复制到终端输入即可

```
1 source /etc/network_turbo
```

```
root@autodl-container-51bc499713-7af1baf7:~# source /etc/network_turbo
设置成功
注意：仅限于学术用途，不承诺稳定性保证
root@autodl-container-51bc499713-7af1baf7:~#
```

4.2 模型管理平台中部署Ollama

没有ollama之前，部署一个私有的大模型比较复杂。需要在主机上安装很多依赖的软件。但现在我们只需要安装Ollama，大模型相关的依赖，Ollama都提供好了。

Ollama介绍

当涉及到大型语言模型（LLMs）的管理、部署和使用，使用像 Ollama、XInference 和 LocalAI 这些模型管理工具可以显著提升模型的可管理性，提高工作效率。

Ollama是在Github上的一个开源项目，项目定位是：**一个本地运行大模型的集成框架**。通过将模型权重、配置文件和必要数据封装进由 **Modelfile** 定义的包中，从而实现大模型的下载、启动和本地运行的自动化部署及推理流程。

Ollama官方地址：<https://ollama.com>

Ollama Github开源地址：<https://github.com/ollama/ollama>

Ollama项目支持跨平台部署，目前已兼容Mac、Linux和Windows操作系统。无论使用哪个操作系统，Ollama项目的安装过程都设计得非常简单。

① Ollama的部署(下载-启动)

使用连接工具打开我们用于部署AutoDL上的GPU服务器 Ollama

启动学术加速

```
1 source /etc/network_turbo
```

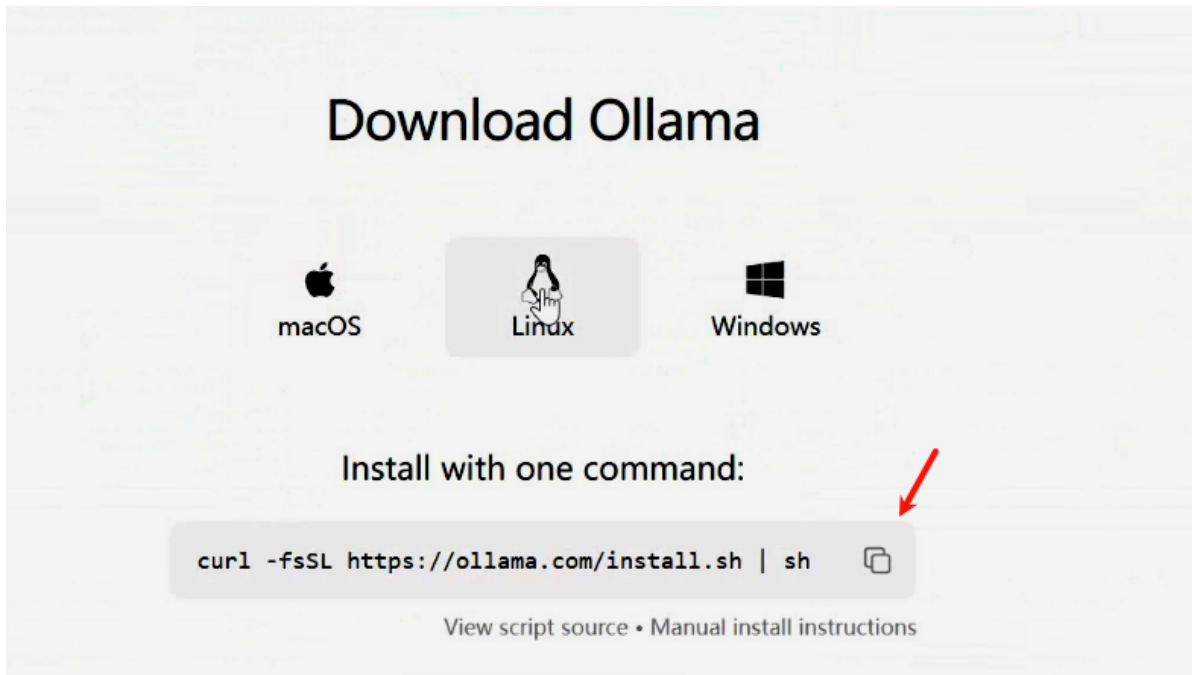
来到Linux如下目录

```
1 cd /data/dify/ #没有此目录, 可以先创建。 sudo mkdir ./data 等
```

执行命令

```
1 curl -fsSL https://ollama.com/install.sh | sh
```

上述指令来自于:



下载完毕

```
root@autodl-container-51bc499713-7af1baf7:~# curl -fsSL https://ollama.com/install.sh | sh
>>> Installing ollama to /usr/local
>>> Downloading Linux amd64 bundle
##### 100.0%
>>> Creating ollama user...
>>> Adding ollama user to video group...
>>> Adding current user to ollama group...
>>> Creating ollama systemd service...
WARNING: systemd is not running
WARNING: Unable to detect NVIDIA/AMD GPU. Install lspci or lshw to automatically detect and install GPU dependencies.
>>> The Ollama API is now available at 127.0.0.1:11434.
>>> Install complete. Run "ollama" from the command line.
root@autodl-container-51bc499713-7af1baf7:~#
```

启动Ollama服务

```
1 OLLAMA_HOST=127.0.0.1:6006 ollama serve
```

② Ollama中下载大模型

地址: <https://ollama.com/library> 查看可以部署哪些模型

举例: 接入Qwen3:4b开源模型

qwen3

ollama run qwen3

3.6M Downloads Updated 1 month ago

Qwen3 is the latest generation of large language models in Qwen series, offering a comprehensive suite of dense and mixture-of-experts (MoE) models.

tools thinking 0.6b 1.7b 4b 8b 14b 30b 32b 235b

Models

View all →

Name	Size	Context	Input
qwen3:latest	5.2GB	40K	Text
qwen3:0.6b	523MB	40K	Text
qwen3:1.7b	1.4GB	40K	Text
qwen3:4b	2.6GB	40K	Text
qwen3:8b	5.2GB	40K	Text
qwen3:14b	9.3GB	40K	Text
qwen3:30b	19GB	40K	Text
qwen3:32b	20GB	40K	Text
qwen3:235b	142GB	40K	Text

复制该指令

qwen3:4b

ollama run qwen3:4b

3.6M Downloads Updated 1 month ago

Qwen3 is the latest generation of large language models in Qwen series, offering a comprehensive suite of dense and mixture-of-experts (MoE) models.

tools thinking 0.6b 1.7b 4b 8b 14b 30b 32b 235b

Updated 1 month ago		2bfd38a7daaf · 2.6GB
model	arch qwen3 · parameters 4.02B · quantization Q4_K_M	2.6GB
template	{{- \$lastUserIdx := -1 -}} {{- range \$idx, \$msg := .Messages -}} {{...}}	1.7kB
license	Apache License Version 2.0, January 2004	11kB
params	{ "repeat_penalty": 1, "stop": ["< im_start >", "< im_end >"], "te	120B

再开启一个终端，执行运行命令。会从网络下载该模型：

```
1 source /etc/network_turbo #先启动学术加速
```

```
1 OLLAMA_HOST=127.0.0.1:6006 ollama run qwen3:4b
2
3 #如果下载过慢，大家也可以下载较小的模型，比如：
4 OLLAMA_HOST=127.0.0.1:6006 ollama run qwen2.5:1.5b
```

安装成功后可以通过提问检验是否成功

```
root@autodl-container-f0054fa37c-a6cf81b7:~# OLLAMA_HOST=127.0.0.1:6006 ollama run qwen2.5:1.5b
pulling manifest
pulling 183715c43589: 20% | 200 MB/986 MB
C
root@autodl-container-f0054fa37c-a6cf81b7:~# OLLAMA_HOST=127.0.0.1:6006 ollama run qwen2.5:1.5b
pulling manifest
pulling 183715c43589: 100% | 986 MB
pulling 66b9ea09bd5b: 100% | 68 B
pulling eb4402837c78: 100% | 1.5 KB
pulling 832dd9e00a68: 100% | 11 KB
pulling 377ac4d7aeef: 100% | 487 B
verifying sha256 digest
writing manifest
success
>>> 你好
你好！很高兴为您服务。有什么我可以帮您吗？

>>> 你是什么模型？
我是阿里云自主研发的超大规模语言模型，我叫通义千问。它能够回答问题、创作文字但可能会出现偶尔的失误，请见谅。如果您有任何问题或需要帮助的地方，随时告诉我！

>>>
```

然后使用 **Ctrl+d** 退出该对话。

查看大模型的安装情况：

```
1 OLLAMA_HOST=127.0.0.1:6006 ollama list
```

③ Dify配置大模型前打通隧道

需要在GPU服务器上的服务已启动的前提下，使用隧道工具在Dify所在的服务器打通隧道



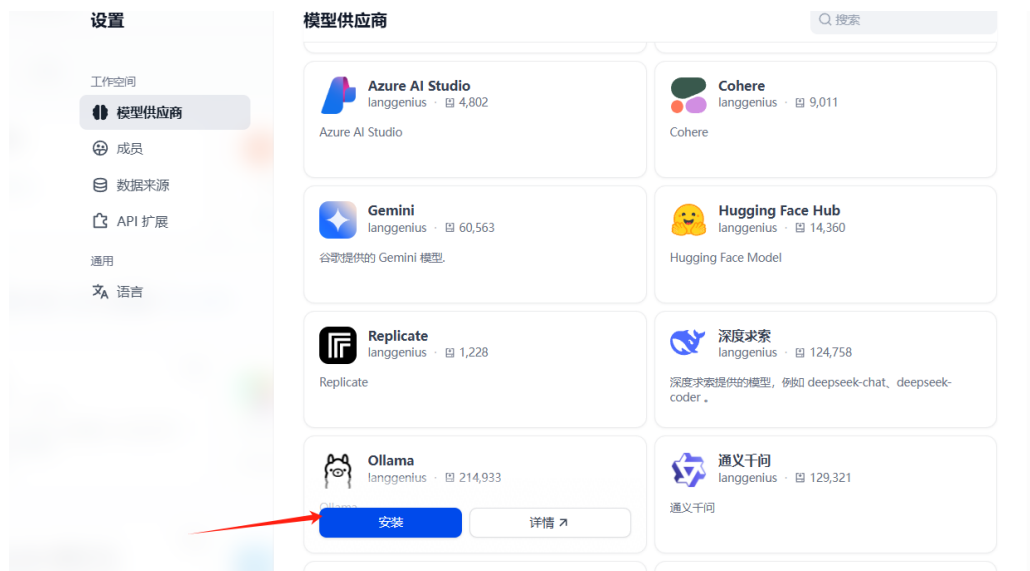
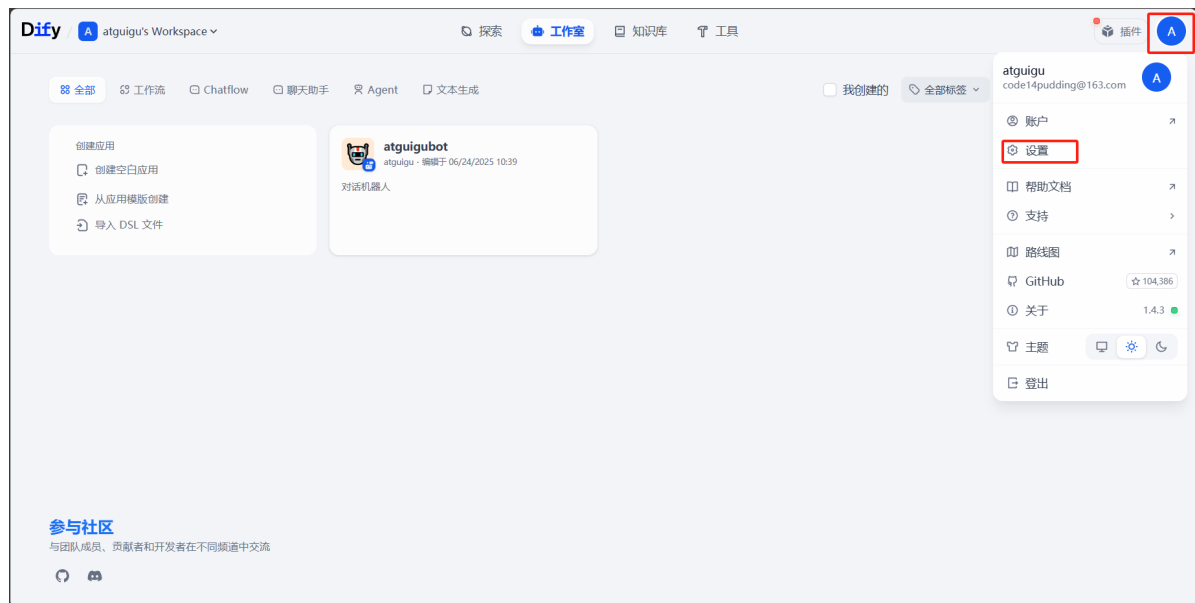
将第一行指令复制，另开一个腾讯云的dify的XShell会话终端，输入该指令。之后按照要求把代码复制粘贴进去，没有任何响应即为成功打通隧道。

```
+-----+
*注意：
1.系统盘较小请将大的数据存放于数据盘或文件存储中，重置系统时数据盘和文件存储中的数据不受影响
2.清理系统盘请参考：https://www.autodl.com/docs/qal/
3.终端中长期执行命令请使用screen等工具开后台运行，确保程序不受SSH连接中断影响：https://www.autodl.com/docs/daemon/
root@autodl-container-f0054fa37c-a6cf81b7:~# ssh -CNg -L 6006:127.0.0.1:6006 root@connect.bjb1.seetacloud.com -p 43960
Warning: Permanently added '[connect.bjb1.seetacloud.com]:43960' (ED25519) to the list of known hosts.
root@connect.bjb1.seetacloud.com's password:
Permission denied, please try again.
root@connect.bjb1.seetacloud.com's password: _____
```

④ Dify中接入Ollama中的大语言模型

在网页Dify页面进行操作：

设置-模型供应商-安装ollama服务

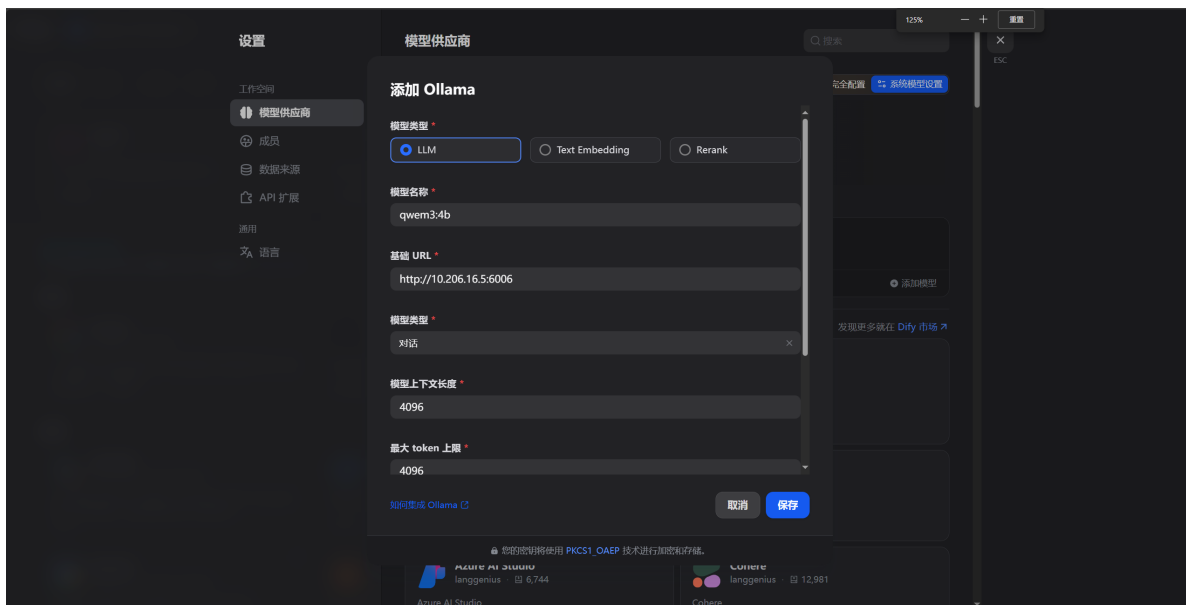


配置Ollama服务的模型



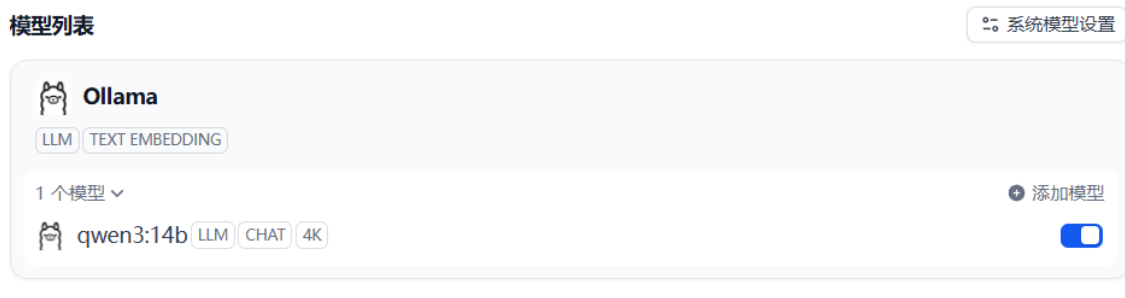
模型名称填写刚刚下载的qwen3:4b

URL中IP地址使用Dify服务器的内网IP即可，端口号使用6006。点击保存

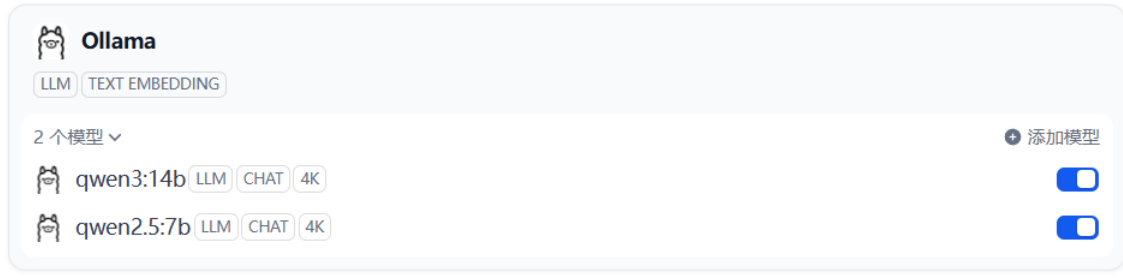


注：当显示添加成功后，但是没有模型的话，说明还未加载成功，并不是配置失败。需要等待一段时间即可刷新出来。配置失败是不会显示配置成功的。

如图添加完毕



可以用相同的方法添加更多模型，比如这里可以在Ollama中下载一个qwen2.5:7b，用相同的方法接入到Dify中



5、主线3：嵌入&重排序模型的部署

5.1 模型管理平台中部署XInference

XInference介绍

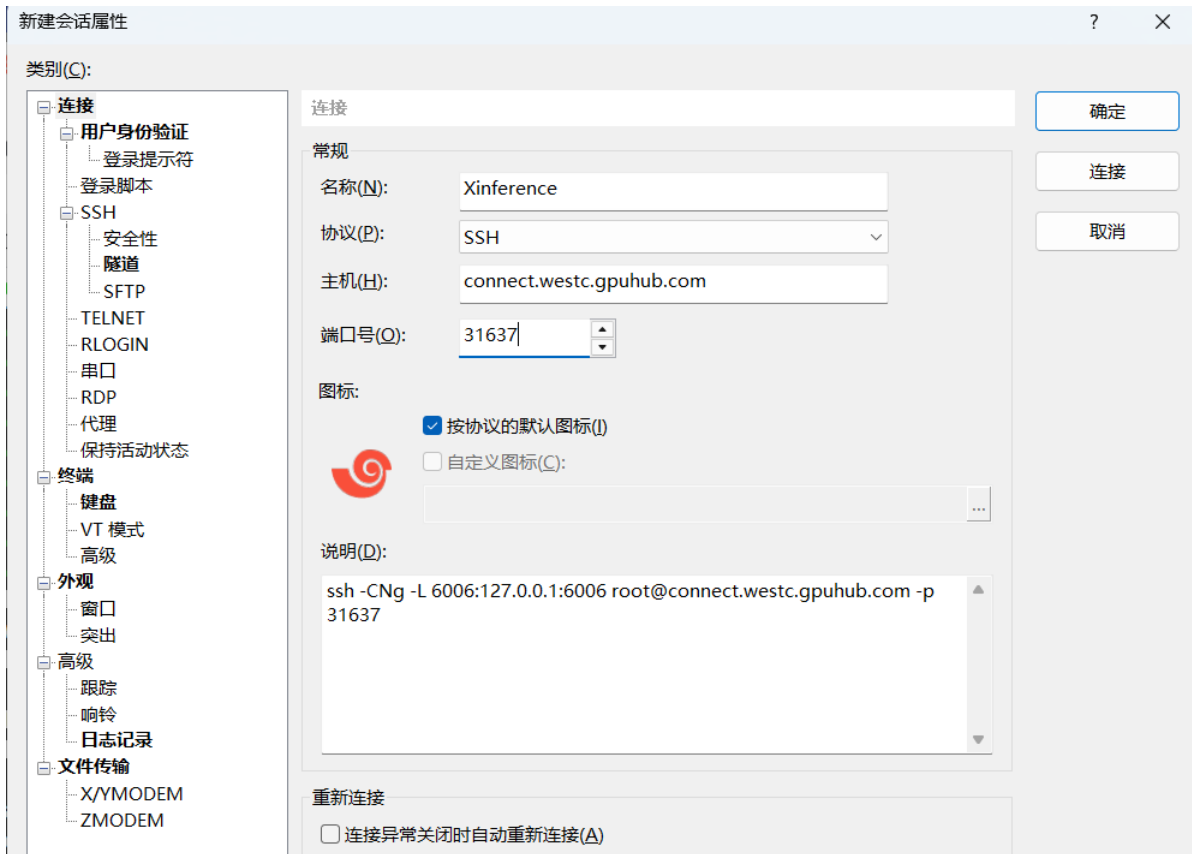
XInference 是一个强大且通用的分布式推理框架，也可以用于私有化部署和运行大语言模型，通过 XInference 可以简化各种AI模型的运行和集成，使开发者可以使用任何开源大语言模型、推理模型、多模态模型在云端或本地环境中运行、推理，创建强大的AI应用。

① XShell登录服务器

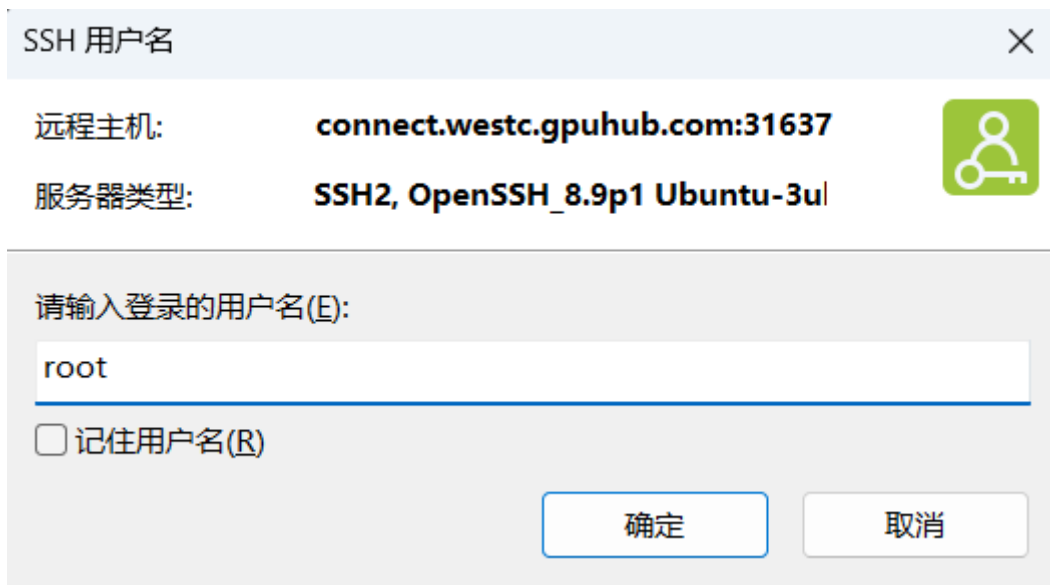
地址参考如下：



粘贴到XShell中进行登录：



用户名默认为root:



密码:



登录成功:

```
1 tencent-dify x 2 tencent-dify x 3 ollama x 4 ollama x 5 Xinference x +
起保存。
||/root/autodl-tmp ||数据盘||快||实例关机数据不会丢失，可存放读写IO要求高的数据。但
不会随保存镜像一起保存||
CPU : 12 核心
内存 : 90 GB
GPU : NVIDIA GeForce RTX 4090, 1
存储:
  系统盘 / : 1% 52M/30G
  数据盘 /root/autodl-tmp: 1% 24K/50G
+-----+
*注意:
1.系统盘较小请将大的数据存放于数据盘或文件存储中，重置系统时数据盘和文件存储中的数
据不受影响
2.清理系统盘请参考: https://www.autodl.com/docs/qa1/
3.终端中长期执行命令请使用screen等工具开后台运行，确保程序不受SSH连接中断影响: http
s://www.autodl.com/docs/daemon/
root@autodl-container-b94c46a1f4-ab02ea99:~#
```

② Xinference的部署(下载)

开启学术加速

```
1 source /etc/network_turbo
```

依次输入如下指令，开始安装Xinference

注意：最新版本会有一些依赖上的冲突，所以下载如下这个版本

```
1 python -m pip install --upgrade pip
2
3 pip install 'xinference==1.5.0'
4
5 #安装torch
6 pip install -i https://pypi.tuna.tsinghua.edu.cn/simple torch
7
8 #升级PyTorch
9 pip install --upgrade torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu118
```

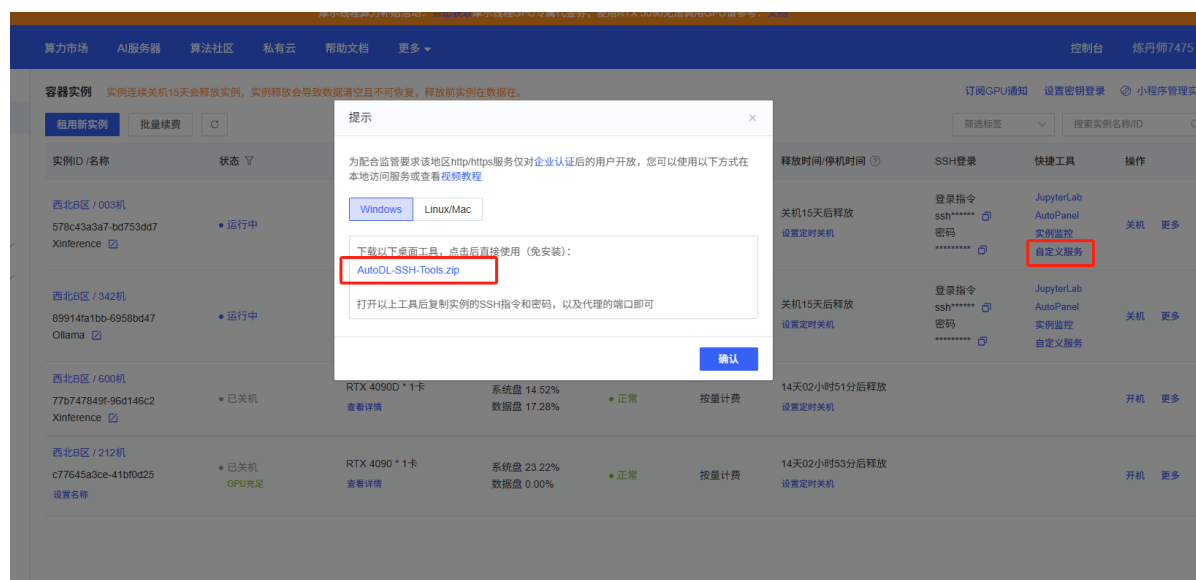
```
1 #启动服务
2 xinference-local --host 0.0.0.0 --port 6006
```

```
6006
2025-08-12 00:49:51,350 xinference.core.supervisor 1572 INFO      Xinference supervi
sor 0.0.0.0:22504 started
2025-08-12 00:49:51,467 xinference.core.worker 1572 INFO      Starting metrics expor
t server at 0.0.0.0:None
2025-08-12 00:49:51,470 xinference.core.worker 1572 INFO      Checking metrics expor
t server...
2025-08-12 00:49:53,444 xinference.core.worker 1572 INFO      Metrics server is star
ted at: http://0.0.0.0:43647
2025-08-12 00:49:53,444 xinference.core.worker 1572 INFO      Purge cache directory:
/root/.xinference/cache
2025-08-12 00:49:53,446 xinference.core.worker 1572 INFO      Connected to superviso
r as a fresh worker
2025-08-12 00:49:53,471 xinference.core.worker 1572 INFO      Xinference worker 0.0.
0.0:22504 started
2025-08-12 00:49:59,105 xinference.api.restful_api 1545 INFO      Starting Xinferenc
e at endpoint: http://0.0.0.0:6006
2025-08-12 00:49:59,231 uvicorn.error 1545 INFO      Uvicorn running on http://0.0.0
.0:6006 (Press CTRL+C to quit)
```

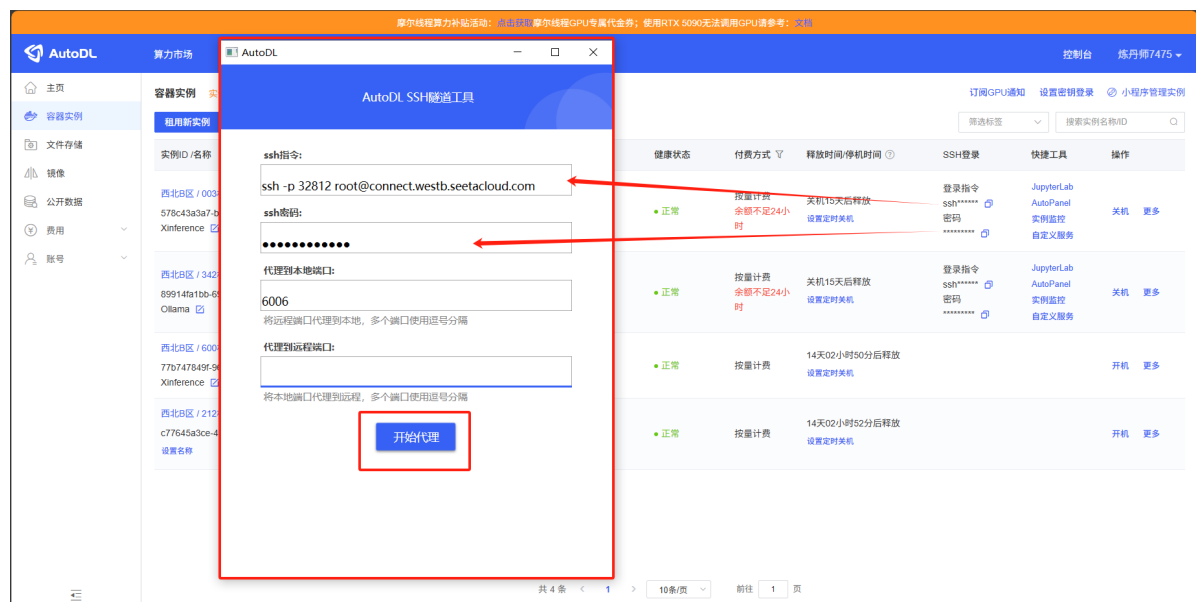
5.2 XInference中下载大模型

① Windows平台访问

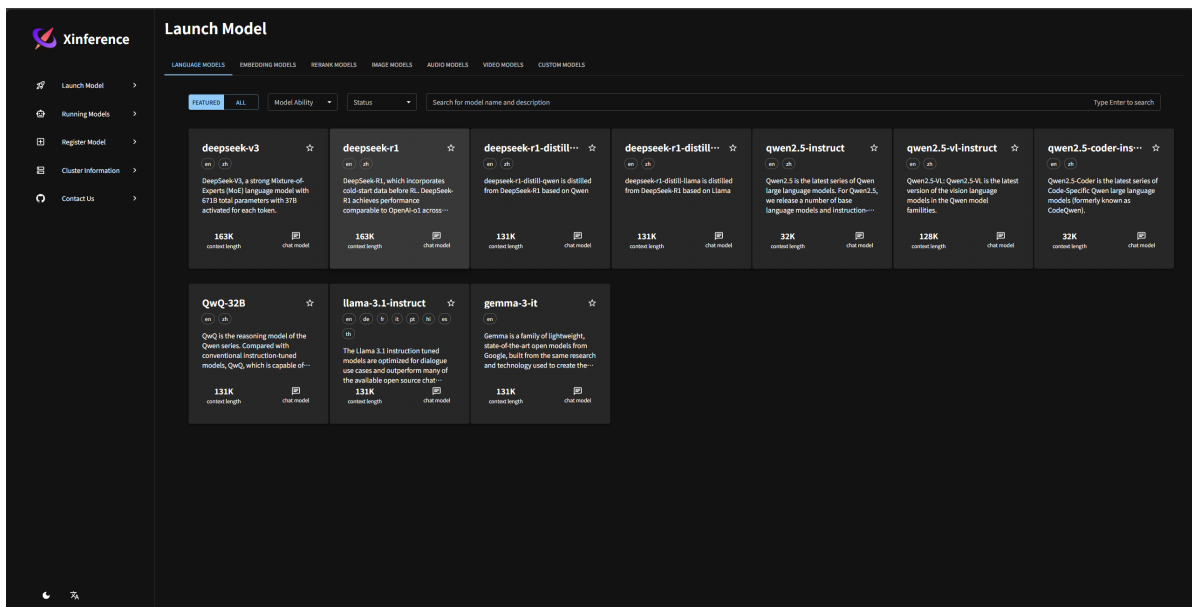
如此服务便启动了，我们可以通过使用Windows的AutoDL隧道工具对其进行访问。下载这个工具



打开该工具，ssh指令和密码粘贴进去，代理到本地端口填写6006

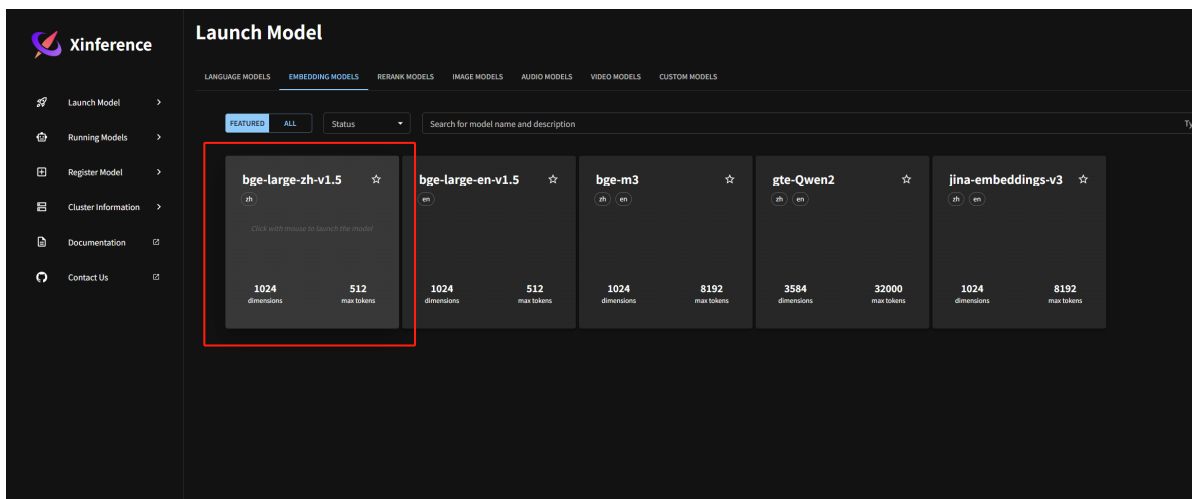


访问<http://127.0.0.1:6006/>



② 模型下载(嵌入+重排序模型)

选择 **嵌入模型**，下载 **bge-large-zh-v1.5**



按照如下配置进行下载模型

bge-large-zh-v1.5

(Optional) Model UID, model name by default

Replica
1

Device
GPU

GPU Idx, Specify the GPU index where the model is located

Worker Ip, specify the worker ip where the model is located in a distributed scenario

(Optional) Download_repo
huggingface

(Optional) Model Path, For PyTorch, provide the model directory. For GGML/GGUF, provide the model file path.

Additional parameters passed to the inference engine

Launch

可能会出现兼容性问题报错，原因是 pip 默认安装的 **xoscar v7.0+** 版本api与 **xinference** 不兼容，降级到 **xoscar v0.6.2** 后可以正常启动。

```
Server error: 500 - [address=0.0.0.0:51032, pid=12889] MainActorPool.append_sub_pool() got an unexpected keyword argument 'start_method'
```

Xinference

Launch Model

Running Models

Register Model

Cluster Information

Contact Us

Language Models **EMBEDDING MODELS** **RERANK MODELS** **IMAGE MODELS** **AUDIO MODELS** **VIDEO MODELS** **CUSTOM MODELS**

FEATURED ALL Status Search for model name and description Type Enter to search

bge-large-zh-v1.5 ☆
zh 1024 dimensions 512 max tokens

bge-large-en-v1.5 ☆
en 1024 dimensions 512 max tokens

bge-m3 ☆
zh en 1024 dimensions 8192 max tokens

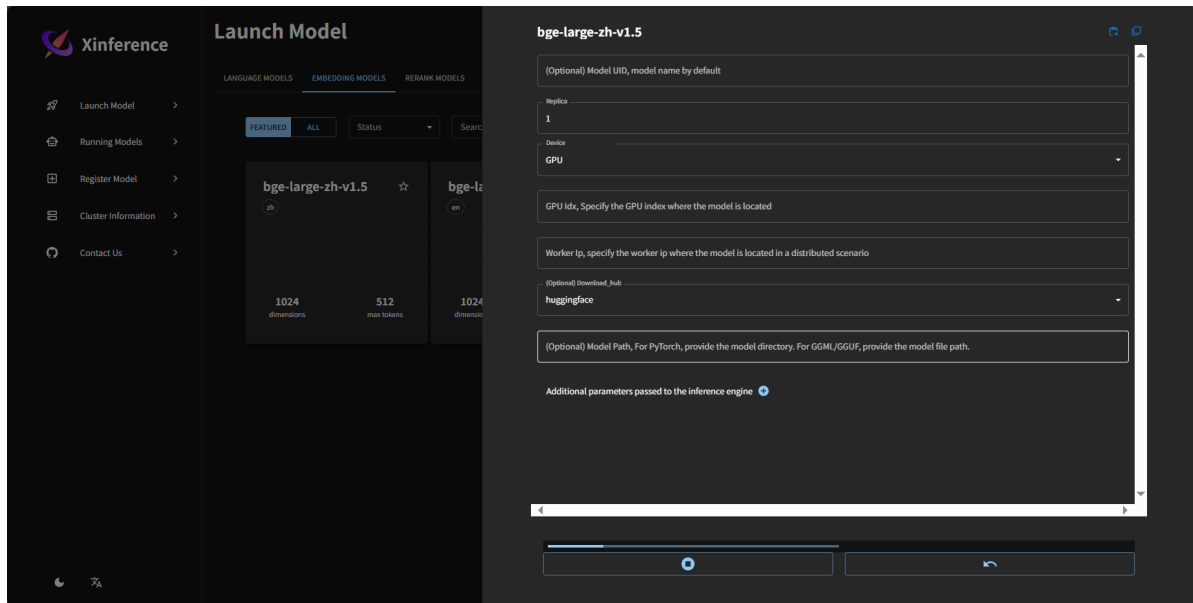
gte-Qwen2 ☆
zh en

jina-embeddings-v3 ☆
zh en

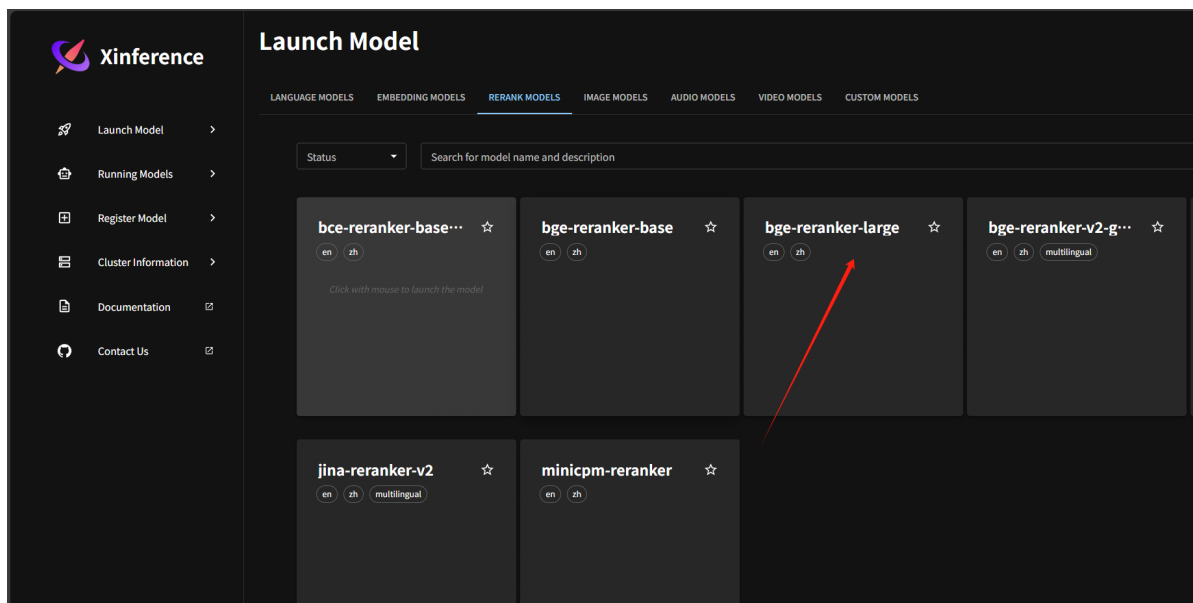
- 1 pip install xoscar==0.6.2
- 2
- 3 #同时安装一些重要的依赖
- 4 pip install sentence-transformers
- 5 pip install sentencepiece transformers torch

之后重启Xinference服务，即可正常下载

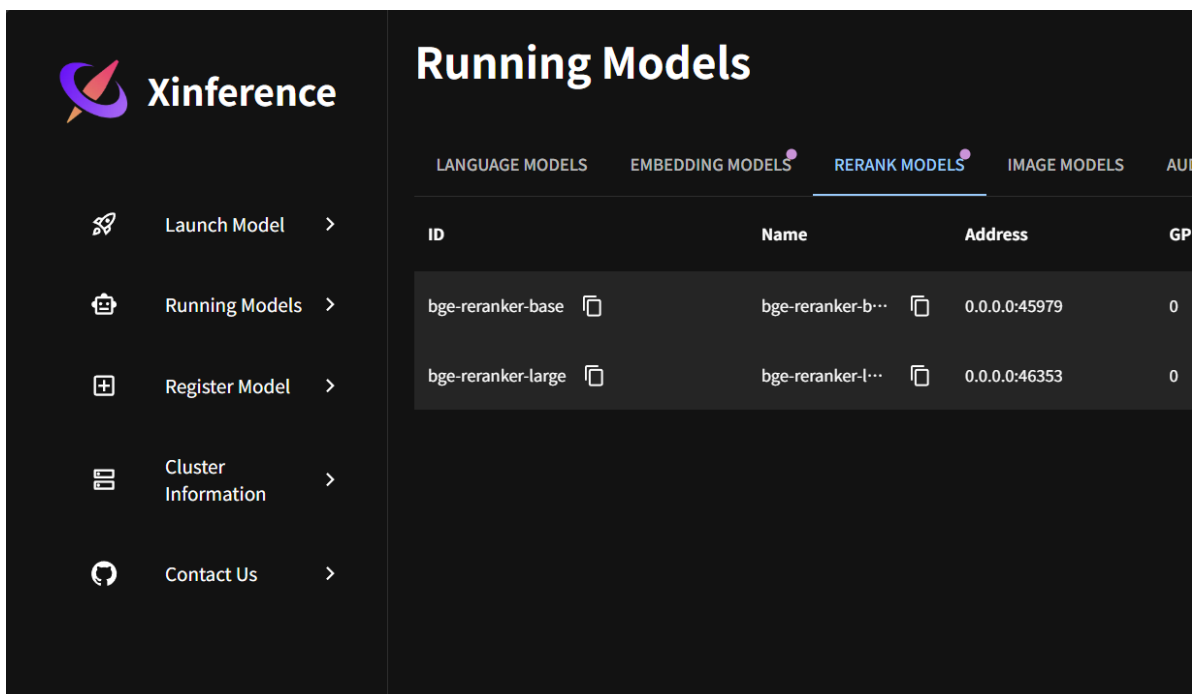
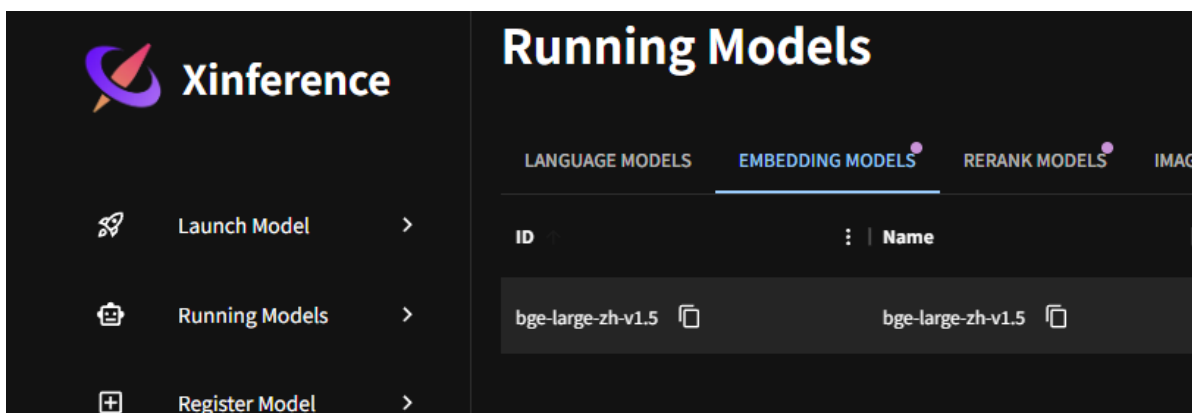
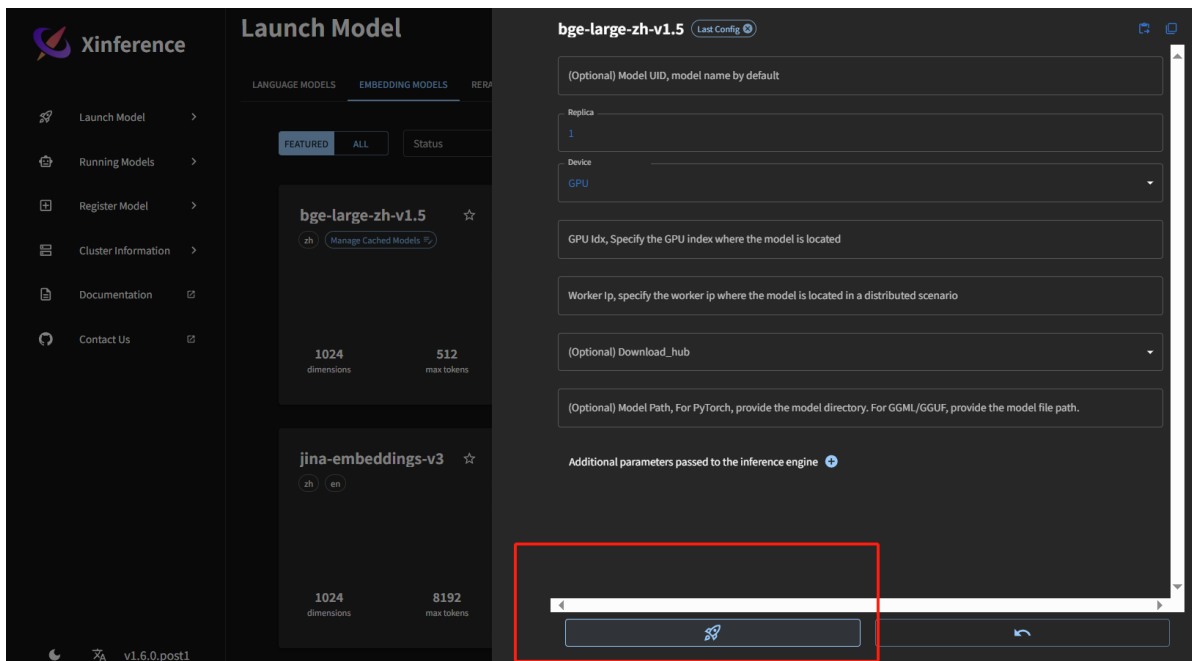
- 1 #启动服务
- 2 xinference-local --host 0.0.0.0 --port 6006



接着，再安装一个重排序模型：



模型安装好了之后，通过launch运行，运行好的模型会在Running Model中看到



③ Dify配置大模型前打通隧道

另起一个Dify服务器XShell会话，使用隧道工具，但是注意把本机端口号改为6007，因为6006被Ollama占用了，按照如下的示例去改造，把第一处的**6006**改为**6007**

```
1 ssh-CNng -L 6007:127.0.0.1:6006 root@connect.westcgpuhub.com -p 31637
```

```
1 tencent-dify x 2 tencent-dify x 3 tencent-dify x 4 ollama x 5 ollama x 6 Xinference x 7 < >

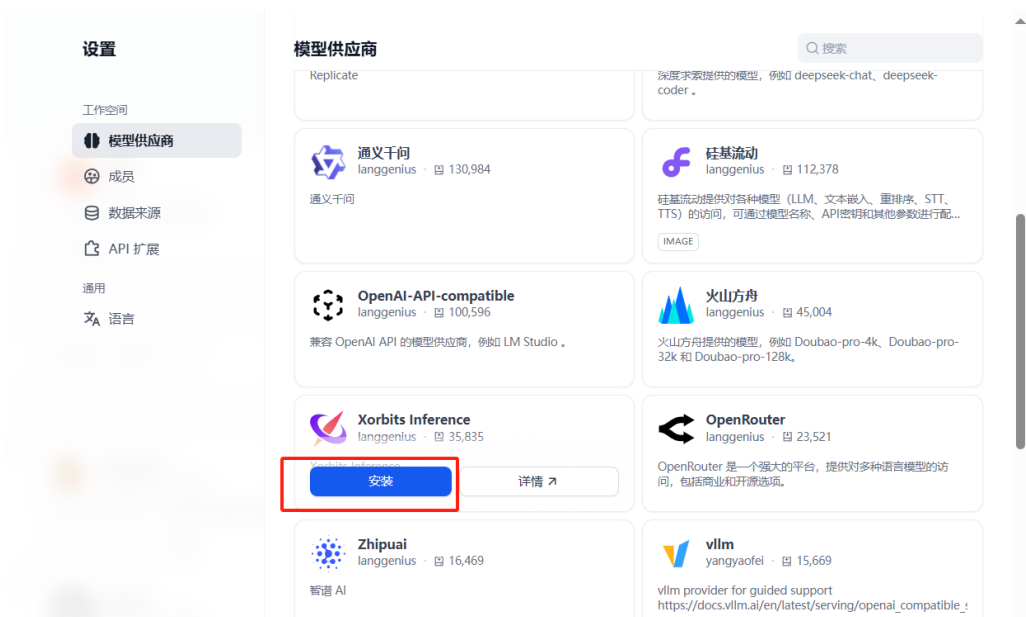
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

*** System restart required ***
Last login: Mon Aug 11 23:41:30 2025 from 120.245.114.200
ubuntu@VM-0-6-ubuntu:~$ cd "/opt/dify/docker"
ubuntu@VM-0-6-ubuntu:/opt/dify/docker$ ssh -CNg -L 6007:127.0.0.1:6006 root@connect
.westc.gpuhub.com -p 31637
The authenticity of host '[connect.westc.gpuhub.com]:31637 ([116.172.66.186]:31637)'
can't be established.
ED25519 key fingerprint is SHA256:fXmxJm1D+u8+mkd+cTJvamLBrxcQyH7IpnvK/lR3v5t8.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[connect.westc.gpuhub.com]:31637' (ED25519) to the list
of known hosts.
root@connect.westc.gpuhub.com's password: _____
```

④ Dify接入Xinference中的模型

在网页Dify页面进行操作：

设置-模型供应商-连接Xinference



按照如下方法将Embedding模型和Reranker模型进行添加

Text Embedding

Rerank

Speech2text

TTS

模型类型选择

模型名称 *

bge-large-zh-v1.5

粘贴模型名称

服务器URL *

http://10.206.0.16:6007

使用Dify服务器内网ip及6007端口

模型UID *

bge-large-zh-v1.5

粘贴模型ID

API密钥

在此输入您的API密钥

如何部署 [Xinference](#)

取消

保存

这里的名称和id要和Xinference平台的保持一致。地址使用http://[dify服务器的内网ip]:6007

添加成功:

工作空间

模型供应商

成员

数据来源

API 扩展

通用

语言

模型列表

系统模型设置

Ollama

LLM | TEXT EMBEDDING

2 个模型

qwen3:14b LLM | CHAT | 4K

qwen2.5:7b LLM | CHAT | 4K

Xorbits Inference

LLM | TEXT EMBEDDING | RERANK | SPEECH2TEXT | TTS

2 个模型

bge-reranker-base RERANK

bge-large-zh-v1.5 TEXT EMBEDDING | 512

